Large Language Models for Quantum Software Engineering: Opportunities and Challenges in Modeling, Generation, Testing, and Repair

JIAMING YE, Southwest Jiaotong University, Chengdu, China
QIANG HU, Tianjin University, Tianjin, China
FUYUAN ZHANG, Zhejiang University, Hangzhou, China
SHANGZHOU XIA and XIAOYU GUO, Kyushu University, Fukuoka, Japan
XIONGFEI WU, University of Luxembourg, Luxembourg, Luxembourg
YONGQIANG LYU, Tianjin University, Tianjin, China, Joint Research Center for System Security, Tsinghua University, Beijing, China, and Science City (Guangzhou) Digital Technology Group Co., Ltd., Guangzhou, China
JIANJUN ZHAO, Kyushu University, Fukuoka, Japan

Large Language Models (LLMs) have already reshaped many aspects of traditional software engineering, from code generation and debugging to automated testing and maintenance. As an important research subdomain in the software engineering area, quantum software engineering (QSE) is poised to undergo a similar transformation. As QSE moves toward broader adoption, including the generation of quantum code, program testing, and automated program repair, LLMs will likely play a key supporting role in its development. However, despite encouraging early attempts, the community still lacks a roadmap for using LLMs in QSE. Without such a roadmap, opportunities may be missed, challenges may remain unrevealed, and risks may be overlooked. In this article, we present our vision for how LLMs can advance QSE, focusing on four core tasks: program modeling and analysis, program generation, testing, and repair. We outline the opportunities and unique challenges, identify the risks inherent in this integration, and propose a research agenda to guide the community toward realizing the full potential of LLM-powered QSE.

Jiaming Ye is with the Manufacturing Industry Chain Collaboration Industrial Software Key Laboratory of Sichuan Province, Chengdu, Sichuan, 610031, China
Fuyuan Zhang is with the State Key Laboratory of Blockchain and Data Security, Zhejiang University, Hangzhou, China.
This work was supported in part by the Fundamental Research Funds for the Central Universities under Grant 2682026CX206, the National Natural Science Foundation of China under Grants U23B2041 and U24A6009, and the Joint Research Center for System Security, Tsinghua University (Institute for Network Sciences and Cyberspace)—Science City (Guangzhou) Digital Technology Group Co., Ltd.; and in part by JSPS KAKENHI under Grants JP26K02892 and JP24K14908, JST SPRING under Grant JPMJSP2136, and JST BOOST under Grant JPMJBS2406.
Authors' Contact Information: Jiaming Ye, Southwest Jiaotong University, Chengdu, China; e-mail: yejiaming@swjtu.edu.cn; Qiang Hu (corresponding author), Tianjin University, Tianjin, China; e-mail: qianghu@tju.edu.cn; Fuyuan Zhang, Zhejiang University, Hangzhou, China; e-mail: fuyuanzhang@163.com; Shangzhou Xia, Kyushu University, Fukuoka, Japan; e-mail: xiaszore19@gmail.com; Xiaoyu Guo, Kyushu University, Fukuoka, Japan; e-mail: guoxiaoyu777@gmail.com; Xiongfei Wu, University of Luxembourg, Luxembourg, Luxembourg; e-mail: xiongfei.wu.a94@gmail.com; Yongqiang Lyu, Tianjin University, Tianjin, China, Joint Research Center for System Security, Tsinghua University, Beijing, China, and Science City (Guangzhou) Digital Technology Group Co., Ltd., Guangzhou, China; e-mail: lyuyq@tju.edu.cn; Jianjun Zhao, Kyushu University, Fukuoka, Japan, e-mail: zhao@ait.kyushu-u.ac.jp.



This work is licensed under Creative Commons Attribution International 4.0.

© 2026 Copyright held by the owner/author(s).
ACM 1557-7392/2026/9-ART273
https://doi.org/10.1145/3828167

CCS Concepts: • **Software and its engineering → Software development techniques**; **Software verification and validation**; • **Computing methodologies → Artificial intelligence**;

Additional Key Words and Phrases: quantum software engineering, roadmap, large language model, quantum software testing, quantum software repair

ACM Reference format:
Jiaming Ye, Qiang Hu, Fuyuan Zhang, Shangzhou Xia, Xiaoyu Guo, Xiongfei Wu, Yongqiang Lyu, and Jianjun Zhao. 2026. Large Language Models for Quantum Software Engineering: Opportunities and Challenges in Modeling, Generation, Testing, and Repair. *ACM Trans. Softw. Eng. Methodol.* 35, 9, Article 273 (September 2026), 42 pages.
https://doi.org/10.1145/3828167

1 Introduction

The birth of ChatGPT has attracted the attention of not only AI researchers but also a wide range of the public by offering a human-like interaction, which facilitates the use of intelligent AI models [99]. Since then, a list of well-known **Large Language Models (LLMs)** has been proposed, such as Claude, Gemini, Llama, and DeepSeek [98]. As one of the impacts of this technological revolution, LLMs have changed **Software Engineering (SE)** techniques. For example, currently, LLMs are widely adopted in software engineering tasks such as program debugging [37], code optimization [74], repository documentation [16], code generation [66], code completion [67], code summarization [3], code translation [81, 104], and lifecycle management of AI models [51]. In short, LLMs have brought significant changes to the field of SE, influencing nearly every subdomain.

Quantum computing has also become an emerging topic. Google proposed its latest quantum chip, Willow, which demonstrates below-threshold quantum error correction and advances the development of large-scale fault-tolerant quantum computing [2]. As quantum computers begin to tackle complex problems that classical computers struggle with, to bridge the gap between developers and powerful quantum machines, the demand for **Quantum SE (QSE)** [94, 158] is growing. A list of research works has been proposed to serve the development of quantum programs, for example, software testing and debugging [76, 88–90, 153], middleware services [7, 105, 114, 128], programming languages [41, 100, 125]. Meanwhile, advances in QSE are driving critical industries, including cryptography [110], materials science [9], and pharmaceuticals [62], to integrate quantum software techniques to accelerate innovation. In the coming years, as the QSE ecosystem matures, it will become a critical infrastructure for computing, much like other well-established programming languages, such as C and Java. In summary, a new era of QSE is on the horizon.

There are already studies that apply LLMs to QSE and have generated promising results. Guo et al. [47] propose using ChatGPT [99] to repair bugs found by Bugs4Q [159]. They prepared 4 sets of prompts with different levels of richness of descriptions. In their experiment, they found 29 of 38 bugs repaired by ChatGPT. D'Aloisio et al. [22] apply multiple LLM models to explain quantum algorithms. They find that even with a small context, LLMs can help models understand quantum algorithms, demonstrating their potential for QSE. In addition, Dupuis et al. [30] propose to fine-tune existing base LLM models to implement a quantum programming assistant. In general, these early studies demonstrated that LLM could help overcome bottlenecks in QSE.

With the promising results of applying LLMs to QSE, one critical question has begun to emerge in the field: What can LLMs contribute to QSE approaches? In other words, since LLMs have reshaped many SE techniques, how will QSE techniques establish their new methodologies and practical applications in light of these advances? Specifically, to achieve technical innovation, what are the most urgent challenges that need to be addressed in the next 5 years? Looking ahead to SE in 2030,

it is essential to address the above questions. However, a roadmap that provides a systematic review of existing approaches and outlines future research opportunities is still lacking.

To fill this gap, we summarize four application directions that are most promising and practical for applying LLMs to QSE. In this article, we focus on a forward-looking vision of possible directions for applying LLMs for QSE. Generally, LLMs can be applied in QSE in four main aspects: (1) quantum program modeling and analysis; (2) quantum program code generation; (3) assisting in quantum program testing; (4) aiding in quantum program repair. These application directions are not mutually independent and can be organically connected. For instance, quantum program modeling and analysis techniques are fundamental to the other directions. Quantum code generation can facilitate the test case generation for quantum programs, while feedback from using these test cases can enhance the quality of code generation. Similarly, quantum code generation can help generate patches for quantum program repair, and repair reports from quantum programs can improve the accuracy of code generation. For each task, we summarize representative works and their limitations, identify concrete opportunities and the distinctive roles LLMs can play, and analyze the open challenges and risks unique to the quantum domain. We consolidate these into a cross-task challenge taxonomy and a phased research roadmap toward 2030, as shown in Table 5 and Figure 3. We do not claim that the four tasks are exhaustive, and intend this roadmap as a starting point that we invite the community to extend.

In this work, we expand our discussion as follows: In Section 2, we introduce the most fundamental quantum computing background; In Section 3, we roughly summarize the development of LLMs as well as QSE, leading to challenges and opportunities in the new era; In Section 4, we summarize the research challenges and opportunities for LLMs in the quantum program modeling and analysis; In Section 5, we discuss the opportunities, challenges, and risks of applying LLMs to generate quantum program code; In Section 6, we depict the future directions of LLMs for testing quantum programs; In Section 7, we outline the future possibilities of LLMs for repairing quantum programs; In Sections 8 and 9, we summarize the entire roadmap.

2 Background

This section establishes the foundational knowledge necessary for the reader to engage meaningfully with QSE. Before progressing to practical implementation aspects such as quantum circuit design and quantum programming paradigms, this section introduces fundamental concepts, beginning with the theoretical explanations of quantum computation, including quantum bits (qubits), superposition, and entanglement. Finally, the section concludes with a comparative analysis of quantum and classical software, highlighting the unique paradigm shifts required in the development of quantum software.

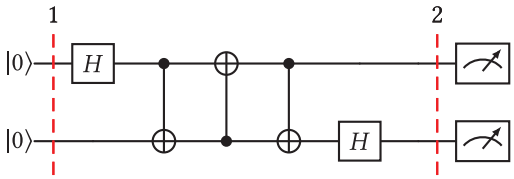
2.1 Qubit and Quantum Circuit

A *quantum bit* (or *qubit*) is the most basic unit in quantum computation. The qubit is the quantum counterpart to the classical bit in classical computers with quantum features. Each qubit must be initialized before being used. In quantum programs, the operations are represented by *quantum logic gates*. The gates provide various functions, including rotating a qubit to an arbitrary angle, assigning superpositions to qubits, and creating controlled connections between qubits. Some gates apply to only one qubit, while others apply to multiple qubits.

The *quantum circuit* is the most basic unit for grouping and managing qubits and quantum gates. A quantum circuit is a model for quantum computation [56], in which the qubits, gates, and measurements are organized in a sequence. As shown in the circuit of Figure 1, a quantum circuit can be divided into three parts: qubits, gate operations, and measurement readouts.

```
1   OPENQASM 2.0;
2   include "qelib1.inc";
3   qreg q[0];
4   qreg q[1];
5   creg c[0];
6   creg c[1];
7   h q[0];
8   cx q[0],q[1];
9   cx q[1],q[0];
10  cx q[0],q[1];
11  h q[1];
12  measure q[0] -> c[0];
13  measure q[1] -> c[1];
```

(a) QASM representation



(b) Circuit representation

Fig. 1. An example of a quantum program written in QASM and its circuit representation.

In the early stages of quantum computing development, quantum programs were composed exclusively of quantum-level logic code, for example, programs written in QASM [10], as shown in the left QASM code in Figure 1. With the rapid development of quantum development languages, especially Python-based Qiskit [141] and Cirq [23], recent quantum programs are a mix of quantum logic and classical code, enabling the implementation of complex program logic. In addition, in previous publications [132, 159], *quantum programs* are considered as combinations of quantum circuits and classical programs.

2.2 Quantum State

In classical programs, the state is represented by a sequence of bits, each with a definite value of either 0 or 1. The most significant difference between quantum bits and classical bits is that quantum bits have *superposition*. The superposition allows a qubit to exist in two orthogonal states simultaneously. For example, for a single qubit program, the state can be written as:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle.$$

The $|0\rangle$ and $|1\rangle$ are the *basis states* of a qubit. These two orthonormal states exist simultaneously, and we cannot tell whether the qubit is definitely in state $|0\rangle$ or in state $|1\rangle$. α and β are the *amplitudes* of the qubit. After measurement, the state of a qubit collapses to 0 or 1, with a certain probability determined by the amplitudes of the superposition. In fact, the measurement probabilities are $|\alpha|^2$ and $|\beta|^2$.

Mathematically, α and β are both complex numbers. If we use the polar representation of a complex number [56], we have the following equivalent representation:

$$|\psi\rangle = a \cdot e^{i\theta}|0\rangle + b \cdot e^{i\phi}|1\rangle.$$

In the more general case of an n-qubit quantum program, assuming $B = \{|0\rangle, |1\rangle, \ldots, |2^n - 1\rangle\}$ to be the set of all basis states, the state of the quantum program can be written as:

$$|\psi\rangle = \sum_{|x\rangle \in B} \alpha_x e^{i\varphi_x}|x\rangle,$$

where x is the measurement value of the basis state $|x\rangle$. In this presentation, α_x and φ_x are the magnitude and phase of $|x\rangle$, respectively.

The *magnitude* and *phase* are quantum features of qubits. Magnitude is an observable property. It is used in quantum computing to measure the probability of observing each state. In quantum

programs, the sum of the squares of all magnitudes of a state equals 1. The square of the magnitude for each state corresponds to the probability of observing that state, e.g., in a single-qubit program, a^2 for $|0\rangle$ and b^2 for $|1\rangle$. Therefore, the magnitude information can be directly reflected in the distribution of measurements.

The phase is a property that cannot be directly measured. To generate a test case with various phase states, we use the Qiskit quantum simulator to simulate the phase of a particular quantum program. In quantum programs, there are two kinds of phase: the *global phase* and the *relative phase*. However, only the relative phase can indicate useful information [56]. So, the phase we discuss in this work refers only to the relative phase; the global phase will not be discussed.

2.3 Entanglement

In quantum programming, *entanglement* refers to the correlation between two or more qubits. Entanglement can be created by applying a multi-qubit gate, such as a CNOT gate, to an appropriate input state, for example, when the control qubit is in a superposition. The resulting qubits may exhibit measurement correlations that cannot be described independently by their individual states. In a large quantum program, entanglement yields rather complex correlations between qubits. In fact, entanglement is a primary feature of quantum programs that distinguishes them from classical programs. Therefore, the entanglement between qubits must be taken into account to generate a diverse set of test cases for quantum programs.

2.4 Statevector

The statevector in a quantum program represents the complete quantum state of a quantum system as a complex-valued vector in the Hilbert space. For an n-qubit quantum system the statevector $|\psi\rangle$ is defined as:

$$|\psi\rangle = \sum_{|x\rangle \in B} A_x |x\rangle,$$

where A_x is the complex probability amplitudes and the normalization condition $\sum_{|x\rangle \in B} |A_x|^2 = 1$ must be satisfied.

In quantum programs, the state vector is computationally represented as an array of 2^n complex numbers, where each index of the array corresponds to a specific basis state x. The amplitude at each index determines the probability of measuring the system in that particular computational basis state, with the measurement probability given by $|A_x|^2$. The statevector is updated via unitary transformations, with each quantum gate performing a linear, norm-preserving operation on the state.

From a quantum program developer's perspective, the statevector is the fundamental data structure for representing quantum states in simulators and quantum virtual machines. The statevector enables precise tracking of quantum superposition and entanglement throughout program execution.

3 QSE in LLM Era: Challenges and Opportunities

3.1 Recent Advances in QSE

As quantum programming languages continue to develop, increasing attention has been paid to the systematic development and maintenance of quantum software, giving rise to the emerging field of QSE [5, 109, 158]. In recent years, QSE has attracted increasing attention as researchers seek to adapt established SE principles and develop new methodologies to address the unique characteristics of quantum software. Unlike classical software, quantum programs are deeply

influenced by quantum mechanical principles, resulting in fundamental differences in program structure, semantic expressiveness, and development paradigms [94].

At the language and development level, researchers are actively designing quantum programming languages and frameworks, such as Q#, Qiskit [141], and Cirq [23], to facilitate quantum application development. However, writing high-quality quantum programs remains difficult: arbitrary quantum programs are often meaningless without algorithmic grounding, and their execution results are inherently non-intuitive. To address these issues, recent work focuses on high-level abstractions, quantum-specific design patterns, and adapted development processes to improve usability [25, 43, 58]. Beyond program construction, quality assurance and lifecycle management have also become central research themes, covering quantum-specific testing, debugging, repair, distributed execution, platform integration, and model-driven engineering [12, 39, 40]. Together, these efforts highlight both the rapid progress and the persistent challenges shaping modern QSE.

These differences significantly hinder the direct application of traditional SE techniques and introduce challenges throughout the entire quantum software lifecycle, including modeling, analysis, generation, testing, and repair. Advancing QSE, therefore, cannot rely solely on classical methodologies; instead, it requires integrating emerging technologies, particularly LLMs, to help developers cope with limited observability, probabilistic behavior, and the high barrier to quantum programming.

3.2 LLMs for SE

Current research on the application of LLMs in SE is growing rapidly. LLMs are widely used in tasks such as code generation [66], code completion [67], code summarization [3], and API recommendation [51]. Models like Codex and ChatGPT can generate code snippets and even solve complex programming problems, significantly increasing coding efficiency and reducing the developer workload [18]. LLMs are also used for code translation [81, 104], allowing conversions between different programming languages. Furthermore, LLMs show promise in code understanding [96] and extracting key information [116]. As LLMs play important roles in many sub-domains of SE, they have significant potential to assist SE across various domains.

LLMs can also be used to analyze software programs and recommend suggestions to strengthen their defenses against external attacks. There is a list of works in which LLMs play an active role, such as vulnerability detection [78], program repair [164], penetration testing [48], and software audits [92]. Specifically, LLMs can identify underlying vulnerabilities in code and provide corresponding repair suggestions, helping developers quickly locate and fix issues. Studies show that LLMs can generate test cases and detect vulnerabilities in applications [20].

LLMs are being applied to enhance existing tools and improve the efficiency of software execution. Specifically, LLMs can understand natural language and convert it into executable code [138, 147, 150], enabling developers to focus more on high-level design rather than low-level implementation details. For example, LLMs can generate code based on users' natural-language instructions or explain complex software features. This makes LLMs useful for building user documentation and providing personalized development suggestions.

In summary, research on the application of LLMs in SE is advancing rapidly, and LLMs are reshaping the way software is managed, developed, and interacted with. Despite the challenges, continued progress in LLM technology is expected to further strengthen its role in SE.

3.3 Early Applications of LLMs in QSE

The application of LLMs to QSE is still in its early stages, but it demonstrates substantial potential. The primary focus is on using LLMs to assist in generating, testing, and repairing quantum code to improve the efficiency and quality of quantum software development. With the growing application

of LLMs in SE, more and more researchers are using them to generate quantum code. And there are studies that help researchers and developers use LLMs more effectively. For example, IBM proposes a benchmark to evaluate LLMs' ability to generate quantum code [129]. The quantum programming assistant is also proposed to help developers build quantum programs [30].

Guo et al. [47] make an early attempt to use ChatGPT to repair bugs in quantum programs. In total, four types of prompts are studied, with different levels of richness of program descriptions. The results of the experiment show that even a subtle hint of the location of the bugs would greatly improve the success rate of the repair. This study systematically investigates the capabilities of LLMs for quantum program repair, empirically validating their effectiveness when explicit task requirements or reward mechanisms are in place. In particular, the experimental results reveal that LLM-based approaches perform closely to manual repair on a benchmark of 38 bugs. As the field of QSE witnesses the rapid development of increasingly powerful language models, the potential of LLM-driven approaches across QSE domains demands systematic investigation by both AI and quantum computing research communities.

4 LLMs for Quantum Program Modeling and Analysis

The analysis of quantum programs centers on extracting rich semantic information that can aid developers in understanding program behavior or facilitate an effective program search. This topic has attracted increasing attention, and several studies have begun investigating related techniques.

Table 1 provides a structured overview of representative studies, key challenges, and emerging opportunities, highlighting the role of LLMs in enhancing semantic understanding, defect detection, and program analysis, while also exposing challenges such as hallucination, scalability limitations, and data scarcity. We next discuss these aspects in detail.

4.1 Existing Challenges in Quantum Program Modeling and Analysis

Existing approaches to quantum program modeling and analysis include requirements engineering, analysis of program entanglement states, bug detectors, and code-analysis language models. In this section, we summarize representative works and discuss their strengths and limitations.

Zhao [158] introduces an early generic quantum software lifecycle covering requirements analysis, design, implementation, testing, and maintenance. Dey et al. [24] present another lifecycle model for quantum software development. Weder et al. [139] further propose an integrated lifecycle for hybrid quantum applications, connecting the development of quantum programs, classical programs, and the workflows that orchestrate them.

At the design and modeling level, several UML-based approaches have also been proposed for quantum software modeling. Pérez-Delgado and Pérez-González [107] introduce UML extensions for representing quantum software, while Pérez-Castillo and Piattini [106] define a UML profile for modeling quantum circuits and hybrid classical–quantum systems. More recently, Guo et al. [45] propose QuanUML, which supports both high-level system modeling and detailed modeling of quantum operations for model-driven quantum software development.

Xia and Zhao [145] propose to analyze the entanglement relationship of a quantum program based on the construction of an interprocedural entanglement graph. They formalize the quantum program code as an operation line to model calls and main operations within the entanglement relations. Beyond entanglement analysis, Yu and Zhao propose the **Quantum Program Dependence Graph (QPDG)**, which extends the classical program dependence graph with quantum-specific dependence relations and supports applications such as program understanding, testing, debugging, and maintenance [155].

Zhao et al. [160] propose QChecker, a static quantum program bug detector. QChecker takes a Qiskit quantum program as input and extracts its attributes and quantum operations from the

Table 1. Existing Challenges in Quantum Program Modeling and Analysis and Future Opportunities by Adopting LLMs

Representative Works and Existing Challenges	Opportunities and Roles of LLMs	Future Challenges
Representative Works: Quantum software lifecycle and development models [24, 139, 158], UML-based quantum software modeling [45, 106, 107], Entanglement and dependence analysis [145, 155]. *Existing Challenges*: Domain knowledge.	*Semantic-Aware Program Summarization and Documentation*: LLMs can capture and explain the high-level intent of quantum programs, overcoming the semantic limitations of pattern-based analysis. *Roles of LLMs*: Mapping abstract circuit semantics to natural language explanations.	*Hallucination of Analysis*: LLMs often hallucinate incorrect interpretations of quantum circuits due to their difficulty in accurately capturing quantum semantics.
	Bug Pattern Summarization: LLMs can learn complex, non-local quantum bug patterns from data, enabling more effective and adaptable defect detection than rule-based tools. *Role*: Assisting data-driven bug discovery.	
Representative Works: Quantum program bug detection and pattern analysis [103, 120, 160], QIR-level code-safety analysis [79]. *Existing Challenges*: Quantum program semantics.	*Enhancing Program Search and Retrieval*: LLMs enable intent-aware semantic search over quantum programs, allowing natural language queries to retrieve relevant code beyond keyword or structural matching. *Roles of LLMs*: Assisting semantic quantum code search.	*Scalability of Program Analysis*: LLMs struggle to analyze quantum programs due to long code representations and exponential state complexity that exceed their context and semantic capacity.
	Scalable Formal Model Generation: LLMs can automatically generate formal representations of quantum programs, improving the scalability of formal analysis methods. *Roles of LLMs*: Assisting formal model generation.	
Representative Works: LLMs for quantum program analysis [83]. *Existing Challenges*: Lack of high-quality quantum program dataset.	*Facilitating Cross-Language Translation for Analysis*: LLMs can bridge heterogeneous quantum languages by learning semantic equivalence, enabling unified cross-platform analysis and translation. *Roles of LLMs*: Semantic analysis assistant.	*Scarcity of Annotated Analysis Data*: LLMs face a lack of high-quality labeled quantum datasets, making synthetic data generation necessary for effective training.

QIR, quantum intermediate representation.

abstract syntax tree of the program. Based on domain knowledge, QChecker is equipped with eight detector patterns. After detection, detailed bug line numbers and bug descriptions are provided. Similarly, Paltenghi and Pradel [103] propose LintQ to support 10 patterns for the analysis of quantum programs. At the intermediate-representation level, Luo and Zhao formalize the data types and instruction set of **Quantum Intermediate Representation (QIR)** to support rigorous code-safety analysis and detect unsafe operations in QIR programs [79].

Shen et al. [120] present a pattern-based quantum program analysis approach that effectively identifies key quantum state constructions, such as uniform superpositions and the creation of entanglement. Their evaluation applies the proposed analysis patterns to a suite of representative quantum–classical hybrid algorithms, demonstrating a precision rate exceeding 90%.

Mao et al. [83] propose applying a pre-trained code model to help capture the semantics of quantum software code and applying few-shot learning to learn effectively from a small number of manually written defect programs. In the test dataset, their model outperforms other baselines.

In summary, lifecycle and UML-based modeling approaches provide structured support for quantum software development and design. At the program-analysis level, Xia and Zhao formalize entanglement relationships, while Yu and Zhao introduce a QPDG to capture broader quantum-specific dependence relations. Luo and Zhao further provide a formal foundation for code-safety analysis at the QIR level. In contrast, QChecker, LintQ, and the approach of Shen et al. rely on manually defined rules or patterns to detect bugs or identify common quantum program structures. Although effective for known cases, these techniques remain limited in capturing complex program semantics and require continuous maintenance. The approach of Mao et al. offers a different perspective by leveraging AI models to assist in analyzing quantum programs, providing promising insights into the automation of semantic understanding. Nevertheless, the lack of large, high-quality datasets remains a critical challenge that limits the robustness and scalability of AI-driven solutions. The challenges of the existing analysis techniques for quantum programs can be summarized as follows.

Existing Challenges of Quantum Program Modeling and Analysis. Effective quantum program analysis relies on deep domain knowledge. Existing approaches use formal entanglement models or pattern-based syntax analysis but suffer from limited semantic scalability and heavy manual maintenance.

4.2 Opportunities of LLMs for Quantum Program Modeling and Analysis

The challenges in current quantum program analysis, which rely on deep domain knowledge and are limited in scalability, create significant opportunities for LLMs to provide automated, scalable, and semantically rich analysis capabilities. LLMs can leverage their understanding of programming structures and textual descriptions to bridge the gap between human intuition and the complex, non-intuitive behaviors of quantum programs. The key opportunities for LLMs in quantum program modeling and analysis are:

(1) *Semantic-Aware Program Summarization and Documentation*: Current pattern-based analysis often only performs syntax-level scanning and struggles with understanding the overall program's semantic purpose, especially in complex, non-intuitive quantum circuits. LLMs, with their advanced natural language processing capabilities, can be trained to summarize the high-level intent and function of a quantum program or a specific circuit segment. D'Aloisio et al. [22] propose a demonstration on applying open source LLMs, e.g., Llama2 and GPT3.5, to explain seven state-of-the-art quantum algorithms. Their experiments show that with a small amount of context, LLMs can produce high-quality explanations of the quantum algorithm. Their work yields a promising result that allows developers to quickly grasp

the program's behavior without needing to interpret complex gate sequences, effectively addressing the domain-knowledge barrier in quantum computing.

Role of LLMs in This Task. Semantic-aware summarization of quantum programs requires producing intent-level natural language explanations. Classical pattern-matching analyses can detect structural features of quantum programs, yet their outputs cannot express high-level semantic purpose [34, 53]. In SE, template-based documentation has similarly been shown to be insufficient for code summarization [63]. Recent evidence further indicates that LLMs can generate coherent explanations of quantum algorithms with some context [22]. Thus, LLMs evolve from trivial assistants to essential components for mapping abstract circuit semantics to natural language explanations.

(2) *Automatic Bug Pattern Summarization*: While existing tools such as QChecker [160] use manually curated patterns, LLMs can move beyond simple rule-based detection. By training on a diverse corpus of correct and defect-prone quantum code, LLMs can learn latent bug patterns that are difficult to formalize manually. This includes identifying non-local errors related to entanglement, improper qubit initialization/reset, or subtle violations of quantum mechanics principles (e.g., attempting to clone an unknown state). Although not yet applied to quantum program analysis, LLMs outperform other classical analyzers in static code analysis and defect detection on test sets of increasing complexity, as reported by Woźny et al. [143]. This capability enables continuous, data-driven updates to analysis patterns, addresses maintenance issues, and improves the detection of sophisticated flaws.

Role of LLMs in This Task: Rule-based quantum bug detectors rely on manually curated patterns and fail to capture non-local semantic defects [160]. LLMs can assist these detectors in continuously learning bug patterns from failure cases and analyzing complex defects [91], making them a necessary data-driven assistant for bug pattern discovery.

(3) *Enhancing Program Search and Retrieval*: When developers search for existing quantum code (e.g., a specific quantum Fourier transform implementation), they often use descriptive, high-level queries. LLMs can be used to create rich vector-based semantic embeddings of quantum programs that capture the intent of the program rather than just its structure. As reported in [123], applying **Retrieval-Augmented Generation (RAG)** to quantum-classical hybrid program code can help LLMs search and generate more accurate results. This enables highly effective semantic search, allowing a developer's natural-language query (e.g., "Find a Qiskit circuit for entanglement swapping") to retrieve the most relevant code, even if keyword usage is sparse or the circuit structure differs from the query. This facilitates code reuse and understanding.

Role of LLMs in This Task. LLMs are fundamentally necessary for semantic quantum code search, as they uniquely bridge the abstract intent of quantum algorithms and their varied implementations. Models like GroverGPT demonstrate excellent quantum-specific understanding [130], while RAG frameworks address the limitations of keyword matching in the face of the high structural variance and hybrid quantum–classical logic inherent to this domain.

(4) *Scalable Formal Model Generation*: Formal models, such as the interprocedural entanglement graph proposed by Xia and Zhao [145] and the QPDG proposed by Yu and Zhao [155], provide structured representations for quantum program analysis. However, constructing, validating, and maintaining such models for large and complex quantum programs remains challenging. LLMs may assist in extracting candidate graph structures and annotations from quantum code, thereby reducing manual effort. Nevertheless, because LLM-generated representations may contain semantic errors, existing formal analyses are still needed to validate their correctness.

Role of LLMs in This Task. LLMs can serve as scalability aids by assisting the extraction and construction of formal program representations. They should complement, rather than replace, formal analysis techniques, which remain necessary for ensuring the correctness of the generated models.

(5) *Facilitating Cross-Language Translation for Analysis*: The quantum ecosystem is fragmented, with programs written in languages such as Q#, Qiskit (Python), and Cirq. LLMs can be trained to understand the semantic equivalence across these different frameworks. This allows an LLM-powered analysis tool to model and analyze code written in one language (e.g., Qiskit) while referencing or comparing it to equivalent concepts in another (e.g., Q#), providing a unified and comprehensive view of a program's behavior regardless of the underlying language. As demonstrated in the experiments of Siavash and Moin [122], LLMs can transform the QSDK language into quantum programming languages on other platforms.

Role of LLMs in This Task. Cross-language quantum analysis requires semantic equivalence across frameworks. Rule-based translators cannot handle API and abstraction differences well. Recent work shows that LLMs can learn cross-framework semantic mappings and translate quantum programs across SDKs [122], making them a necessary assistant for semantic analysis in this task.

4.3 Challenges of Applying LLMs in Modeling and Analysis

Although LLMs offer exciting potential for assisting in quantum program modeling and analysis, their practical application is hindered by the inherent challenges of language models. Unlike classical software analysis, where code semantics are often explicit in syntax, quantum analysis requires interpreting the evolution of quantum states, which can be challenging for current LLMs.

4.3.1 Challenge 1: Hallucination of Analysis. The most critical challenge is the tendency of LLMs to generate plausible yet incorrect interpretations of quantum circuits, a phenomenon known as hallucinations. In classical programming, variable names and control structures often hint strongly at functionality. However, in quantum programs, the semantics of a circuit is defined by the unitary matrix it implements, which cannot be immediately inferred from the gate sequence (e.g., a sequence of Hadamard and CNOT gates could be an entanglement generator or a basis change depending on context). LLMs may fail to extract the semantics of a quantum circuit and hallucinate in their analysis. Recent benchmarking by Guo et al. on the QuanBench [46] dataset reveals that current LLMs frequently produce "semantic errors," achieving an overall accuracy of less than 40% on quantum tasks despite producing syntactically correct code.

4.3.2 Challenge 2: Scalability of Program Analysis. Effective analysis requires feeding the quantum program into the LLM. However, standard representations like OpenQASM can become exceedingly long for practical circuits, potentially exceeding the context window of the LLM or diluting the model's attention over long sequences of simple instructions. Moreover, the semantic complexity is compounded by the exponential growth of the program's true state space (i.e., the state vector or density matrix) as the number of qubits increases. Mikuriya et al. highlight this limitation in their QCoder benchmark, noting that LLMs struggle significantly with quantum domain-specific languages and hardware constraints when relying solely on static text representations of programs, often requiring external simulator feedback to ground their analysis [86].

4.3.3 Challenge 3: Scarcity of Annotated Analysis Data. Training or fine-tuning LLMs for analysis tasks requires high-quality pairs of (Quantum Code, Semantic Summary) or (Quantum Code, Bug Label). However, such data are scarce because explaining the exact semantic function of an arbitrary quantum circuit is a difficult problem (QMA-complete in some contexts). Jern et al. [54] attempt to

address this in their AGENT-Q system. Their work demonstrates that synthetic data generation is currently necessary to bridge this gap, as a real-world, ground-truth-labeled quantum dataset is difficult to find at the scale required for robust model training.

Challenges of Applying LLMs in Quantum Program Modelling and Analysis

Challenge 1: Hallucination of analysis
Challenge 2: Scalability of program analysis
Challenge 3: Lack of labeled analysis data

5 LLMs for Generating Quantum Programs

LLMs are increasingly assisting in the development of QSE, particularly in quantum program generation. Specifically, LLMs can relieve developers of the burden of repetitive programming tasks, allowing them to focus on higher-level quantum algorithm design and problem modeling and to leverage LLMs to complete the remaining implementation. In this section, we will first outline the existing approaches and emerging roles of LLMs in QSE. Based on discussions and findings from the relevant literature, we will highlight the core challenges that current research in QSE must address.

Generally, as external motivations, the increasing number of human experts and domain knowledge, as well as the advancement of scientific insights and quantum theory, can help advance the application of LLMs in the generation of quantum programs. The central task in this paradigm is the core generation process, where the LLM assists in exploring the quantum-specific software space. However, this core process is limited by a key constraint: the lack of high-quality training data, which directly impacts the model's capabilities. In addition to the generation process, the output artifacts must go through a critical validation stage consisting of two key steps: correctness verification and robustness analysis. These steps ensure that the generated program is consistent with the expected behavior and sufficiently robust.

In addition, the paradigm highlights a feedback loop. The insights gained from the validation stage, including identified bugs, inefficiencies, or vulnerabilities, are not merely an endpoint. Instead, they can be fed back to refine the core generation process itself. This iterative process is essential for creating progressively more accurate, efficient, and reliable models, empowering the generation process not only to be a production process but also a learning system.

Table 2 summarizes representative work, key challenges, and emerging opportunities in quantum program generation, highlighting how LLMs enable automated code generation, algorithm design, and workflow construction while facing issues such as correctness gaps, hardware-aware optimization, data scarcity, hidden bugs, and balancing automation with human innovation. The discussions are elaborated in detail in the following sections.

5.1 Existing Challenges in Quantum Program Generation

In general, generating quantum programs involves translating development requirements into executable quantum code. Existing studies explore this translation through several different paths, including direct implementations of quantum algorithms, formalized user input, randomized generation strategies, and LLM-based approaches. In this section, we review representative works from each category and analyze their respective strengths and limitations.

Chen et al. [17] introduce VeriQBench, a benchmark suite that encompasses combinational, dynamic, and sequential quantum circuits. VeriQBench is constructed by instantiating a variety of quantum algorithms, enabling the circuits to scale from small sizes (around 3 qubits) to large ones

Table 2. Existing Challenges in Quantum Program Generation and Future Opportunities by Adopting LLMs

Representative Works and Existing Challenges	Opportunities and Roles of LLMs	Future Challenges
Representative Works: Quantum circuit benchmarking [17]. *Existing Challenges*: Limited diversity of quantum circuits.	*Assisting Building Program from Algorithm to Implementation*: LLMs can generate executable quantum programs from natural language instructions, significantly lowering the barrier to quantum program generation. *Roles of LLMs*: An algorithm-to-implementation translator.	*Bridging the Program Correctness Gap*: Ensuring the semantic correctness of LLM-generated quantum programs remains challenging due to the difficulty of precise specification and the limited scalability of existing verification methods.
Representative Works: UML specifications-based quantum program generation [44]. *Existing Challenges*: Relying on specification rules.	*Assisting Quantum Algorithm Recommendation*: LLMs can assist or automate the design of PQCs for algorithms like VQE and QAOA, reducing reliance on expert knowledge and enabling exploration of new circuit structures. *Role*: A recommendation and generation assistant.	*Exploring the Quantum-specific Software Space*: Optimizing quantum programs with LLMs is challenging due to the vast space of equivalent circuits and the need to account for hardware-specific physical constraints such as noise, topology, and coherence limits. *The Lack of High-Quality Quantum Training Data*: The development of quantum-specific LLMs is fundamentally limited by the scarcity of large-scale, high-quality labeled quantum program datasets.
Representative Works: Quantum test case generation [153]. *Existing Challenges*: Unclear program semantics.	*Assisting Building Hybrid Software Development Workflow*: LLMs can automatically design end-to-end hybrid quantum–classical workflows, streamlining development and accelerating the implementation of quantum applications. *Roles of LLMs*: Assisting in organizing task workflows.	*Reducing Software Failures*: LLMs may introduce subtle, non-local quantum-specific bugs rooted in physical effects, making them difficult to detect and requiring new mechanisms to ensure reliable quantum program generation.
Representative Works: LLMs for quantum program generation [30]. *Existing Challenges*: Lack of high-quality quantum program dataset		*Balancing Automation with Human-Centric Development*: While LLMs can accelerate quantum programming, enabling them to support human-guided innovation and discovery of novel quantum algorithms remains a key challenge.

PQC, Parameterized Quantum Circuit; QAOA, Quantum Approximate Optimization Algorithm; VQE, Variational Quantum Eigensolver.

exceeding 100 qubits. Although the benchmark offers a collection of quantum programs as datasets, its generation process essentially amounts to direct implementations of the underlying quantum

algorithms. Consequently, the diversity and structural richness of the dataset remain limited, as it does not incorporate broader program variations beyond those dictated by the algorithmic templates.

Guo et al. [44] propose generating quantum programs directly from UML specifications. Their approach parses the UML input into a structured graph and separates it into quantum and classical components. These components are then processed using different strategies: classical code is translated through a set of deterministic transformation rules, whereas quantum behavior is mapped into a quantum-specific intermediate representation that can be further compiled into platform-specific quantum programs. This design enables the UML-driven generation of hybrid quantum–classical software while maintaining compatibility with multiple quantum backends.

Ye et al. [153] propose three approaches for generating inputs to test quantum programs: a UCNOT-based generator, an IQFT-based generator, and a random generator. They treat input preparation as the construction of initialization quantum programs. Their experimental results demonstrate that these approaches can produce diverse input states and the corresponding initialization circuits. However, because the generated programs serve solely as test inputs, they remain relatively small in scale. Moreover, to diversify the input states, the generation process introduces substantial randomness, which in turn reduces the readability and structural clarity of the resulting programs.

Dupuis et al. [30] explore a complementary direction by fine-tuning existing code-oriented language models on curated datasets of quantum programs to improve quantum code generation capabilities. They benchmark several models, including CodeLlama [115] and DeepSeek-Coder [42], and show that fine-tuning enables them to achieve a baseline level of correctness and syntactic fluency in generating quantum programs. However, their study highlights a key bottleneck: the scarcity of high-quality, diverse, and semantically rich quantum program datasets, which fundamentally limits the performance ceiling of LLM-based quantum code generators.

In summary, quantum program generation can be viewed as translating high-level requirements into executable quantum code, and existing approaches can be categorized into four types. The first category, exemplified by Chen et al., adheres strictly to pre-defined quantum algorithms. This guarantees correctness, but limits the resulting programs to narrow algorithmic templates. The second category, represented by Guo et al., treats user-provided UML diagrams as formalized requirements and derives quantum code through structured analysis. Although more flexible, this approach relies on substantial manual effort to produce accurate UML specifications. The third category, as demonstrated by Ye et al., introduces controlled randomness into predefined generation patterns to diversify input states, but this often comes at the cost of reduced clarity and readability of the generated circuits. The final category leverages LLMs, such as in the work of Dupuis et al., yet the scarcity of high-quality fine-tuning data remains a significant bottleneck for achieving strong generative performance. In general, each generation approach offers distinct advantages but also faces inherent limitations; their challenges can be summarized as follows.

Existing Challenges of Quantum Program Generation. A significant challenge lies in precisely capturing and formalizing quantum program requirements, which are often abstract, implicit, or expressed at varying levels of granularity. Even when requirements can be clearly specified, translating them into correct and executable quantum program code remains difficult due to the gap between high-level intent and low-level quantum operations.

5.2 Opportunities of LLMs for Quantum Program Generation

Currently, LLMs demonstrate their potential as powerful programming assistants in several key areas by accepting human instructions and generating concrete quantum code.

(1) *Assisting Program Development from Algorithm to Implementation*: LLMs can accept instructions described in natural language (e.g., "Implement Shor's algorithm to factor the integer 21 using Qiskit") and generate executable quantum program code. Recent studies have shown that models like GPT-4 exhibit remarkable capabilities in generating code for frameworks such as Qiskit and Cirq, thus significantly lowering the barrier to entry for quantum programming [30].

Role of LLMs in This Task. Generating quantum programs from high-level algorithmic intent requires mapping natural language specifications to executable code, which rule-based templates cannot generalize across algorithms or SDKs. Recent work shows LLMs can synthesize correct Qiskit and Cirq implementations from textual prompts [49], making LLMs a must for an algorithm-to-implementation translator rather than a mere assistant.

(2) *Assisting Quantum Algorithm Recommendation*: For recent core applications like the **Variational Quantum Eigensolver (VQE)** and the **Quantum Approximate Optimization Algorithm (QAOA)**, the design of the **Parameterized Quantum Circuit (PQC)** is critical. LLMs can recommend or automatically generate structures based on the characteristics of the problem (e.g., the structure of a molecular Hamiltonian), thereby simplifying the design process that relies heavily on expert intuition and potentially exploring novel structures beyond human design [29].

Role of LLMs in This Task. Designing PQC structures for VQE and QAOA relies heavily on expert knowledge and problem-specific insights, in which heuristic templates cannot generalize. Prior work shows automated architecture discovery can outperform human-designed circuits [29]. LLMs can further generalize this discovery by reasoning over problem semantics, acting as a necessary recommendation and generation engine.

(3) *Assisting the Construction of Hybrid Software Development Workflow*: Existing quantum programs often integrate the classical and quantum parts of the program. It is often difficult to design a development workflow manually. Given a generation instruction, LLMs can design a workflow that includes developing a classical program and a quantum program, respectively. The workflow can include not only the construction and execution of the quantum circuit, but also the classical optimizer and data-processing components that drive the entire process, accelerating end-to-end application development from design to practice.

Role of LLMs in This Task. Hybrid quantum–classical applications require tightly coupled workflows between quantum circuits and classical optimizers, whose manual design is complex and error-prone, as demonstrated by VQE-style algorithms [108]. LLMs exhibit effectiveness for workflow synthesis in SE [33] and can serve as a necessary end-to-end workflow organizer for this task.

5.3 Research Challenges: The Uniqueness of the Quantum Domain

Despite the promising outlook, making LLMs reliable, efficient, and secure tools for quantum program generation requires the research community to address a series of challenges. These are not mere translations of problems from classical SE; instead, they are fundamentally different, rooted in the physical principles of quantum computation.

5.3.1 Challenge 1: Bridging the Program Correctness Gap. Although LLMs have demonstrated proficiency in various code generation tasks, LLM-generated programs can still be syntactically correct while containing logical or semantic defects [19, 28]. For traditional programs, developers can mitigate this by refining prompts into rigorous specifications [84] or by employing automated prompt refinement techniques to constrain model behavior [154, 163]. However, in the quantum domain, expressing the intended functionality precisely and in a fully formalized manner becomes

substantially more difficult and subtle. Quantum algorithms involve complex mathematical transformations, probabilistic behaviors, and hardware-dependent constraints that are challenging to capture in natural-language specifications [26]. As a result, even when LLMs can successfully produce executable quantum code, there remains a significant gap between syntactic validity and true semantic correctness.

LLMs may produce incorrect programs; thus, generated programs should be verified to ensure correctness. There is a list of checks to verify the correctness of the program, such as repeated execution and testing. However, for generated quantum programs, inspecting an unknown quantum state in a quantum program, as we do in classical programs, is difficult due to quantum mechanics [142], making it difficult to verify their correctness. Before LLMs were proposed, the research community tried to use equivalence checking [75] and quantum-specific assertions [52] to address this gap. However, previous methods cannot scale to multiple quantum programs due to inefficiency. In short, the first challenge in applying LLMs to quantum program generation is: How can we automatically and efficiently verify the correctness of a generated quantum program?

5.3.2 Challenge 2: Exploring the Quantum-Specific Software Space. For the same quantum computing task, there may exist an exponential number of equivalent circuit implementations. When optimizing classical software, LLMs mainly consider logical metrics such as algorithmic complexity, memory usage, or readability [118]. In contrast, for quantum programs, optimization is first and foremost a physical problem. Due to the limited coherence times of the qubits, the primary goal is to reduce the circuit depth and the number of noise-sensitive operations (such as CNOT gates) to mitigate the effects of decoherence and gate errors [21]. This optimization is a direct struggle against the physical properties of the hardware (qubit lifetimes, gate usability). For example, logically equivalent circuits can have vastly different usability when executed on hardware with different topologies [31]. How can we use LLMs to find an optimal solution in this vast software space that maximizes usability on specific, noisy physical hardware? This may require combining LLMs with optimization strategies and interaction with real or simulated hardware.

5.3.3 Challenge 3: The Lack of High-Quality Quantum Training Data. Unlike classical software, high-quality, labeled datasets of quantum programs are rare. The capability to understand LLM program code in classical SE is built on millions of lines of open source code from platforms like GitHub, providing a massive data repository. On the other hand, the quantum software field lacks such a benchmark. High-quality, validated implementations of quantum algorithms are rare, and many cutting-edge algorithms exist only as mathematical descriptions in academic papers. Existing benchmarks such as QASMBench [64], Bugs4Q [159], VQE [95], Nation et al. [97], MQT [113], and MNISQ [111], while valuable, are limited in scale. This scarcity of training data is the fundamental bottleneck for developing quantum-specific LLMs. The core question is: How can we train powerful, domain-specific LLMs given the current data-scarce reality? Solutions may include innovative data strategies, such as injecting theoretical knowledge to enable models to learn directly from textbooks and papers, or using program synthesis and data augmentation techniques [8] to expand existing small-scale datasets.

5.3.4 Challenge 4: Reducing Software Failures. LLMs may introduce subtle, quantum-specific bugs into the code. Classical software bugs, such as SQL injection or buffer overflows, are typically issues of input handling or memory management. Quantum software bugs, despite those bugs that result from the compilation process, are rooted in improper handling of the physical characteristics of quantum computing. For example, improper handling of the measurement and reset of an ancilla (auxiliary) qubit can introduce errors into the main computation via quantum entanglement or leak critical information. Such errors are non-local. Their source may be far from where the error

manifests, making them difficult to detect using traditional SE approaches. To address this, Campbell et al. [13] propose applying a multi-agent LLM structure to generate quantum software with fewer failures. Their approach retrieves information from the program context and leverages multiple quantum computing experts to decide the generation process. However, although it outperforms previous works, the generation precision can be substantially improved. The challenge remains open: How can we build defense mechanisms to prevent LLMs from introducing silent-failure bugs caused by quantum physical effects?

5.3.5 Challenge 5: Balancing Automation with Human-Centric Development. LLMs can greatly accelerate quantum programming by generating program code. However, the outputs of existing algorithms and frameworks are limited, potentially hindering the discovery of novel quantum approaches. In a domain where progress often relies on breakthroughs such as Shor's or Grover's algorithms, over-relying on program generation may be risky, as it can produce code that has not been seen before. The challenge is therefore not simply to increase efficiency but to design LLM-based systems that preserve human understanding while enabling deeper exploration. Recently, Liang et al. [71] propose using LLMs for discovering new quantum program architectures. Hence, LLMs have the potential to help researchers find new quantum algorithms. However, a key open question remains: how to develop LLM-based quantum programming environments that integrate automation with human-driven innovation? This requires LLMs that not only generate quantum program code but also provide interpretable reasoning grounded in quantum principles, and support interactive exploration of design alternatives.

Challenges of Applying LLMs in Quantum Software Generation

Challenge 1: Bridging the program correctness gap
Challenge 2: Exploring the quantum-specific software space
Challenge 3: Lack of high-quality quantum training data
Challenge 4: Reducing software failures
Challenge 5: Balancing automation with human-centric development

6 LLMs for Testing Quantum Programs

LLMs have already demonstrated substantial potential in numerous domains of traditional SE, significantly enhancing efficiency and quality [3]. Their applications range from software vulnerability detection [78] to automated test case generation [20]. However, when we shift our focus to the emerging field of QSE, the application of LLMs presents a new, more complex set of opportunities and challenges. These arise directly from the fundamental properties of quantum computing, such as superposition, entanglement, and inherent probabilistic features, which render traditional testing techniques insufficient to ensure the reliability of quantum software.

On the one hand, LLMs are adopted in the application of quantum program testing, including the generation of quantum test cases, test oracle construction, and identifying bug patterns of quantum programs; on the other hand, the key challenges in the application should not be overlooked: complexity of quantum properties, the test oracle problem, the sim-to-real gap, and lack of high-quality quantum testing datasets. In this section, we first provide a detailed exploration of the current and potential applications of LLMs in quantum program testing, and then we dive into the key challenges that future research must overcome to realize this potential.

Table 3 summarizes representative work, key challenges, and emerging opportunities in quantum program testing, highlighting how LLMs can enhance test generation, oracle construction, and bug

Table 3. Existing Challenges in Quantum Program Testing and Future Opportunities by Adopting LLMs

Representative Works and Existing Challenges	Opportunities and Roles of LLMs	Future Challenges
Representative Works: Testing criteria [4, 76, 119, 135]. *Existing challenges*: Existing criteria focus on input and output.	*Intelligent Generation of Quantum Test Cases*: LLMs can generate complex quantum test cases as valid quantum circuits, enabling effective testing by leveraging their ability to understand quantum semantics and uncover corner cases. *Roles of LLMs*: Assisting meaningful test case generation.	*Quantum-specific Test Coverage*: Designing effective test coverage for quantum programs remains challenging, requiring LLMs to understand quantum properties and generate minimal test sets that adequately explore the quantum state space.
Representative Works: Quantum program testing technique [1, 50, 55, 75, 77, 102, 131–134, 136, 137, 146]. *Existing Challenges*: Limited effectiveness.	*Assistance in Test Oracle Construction*: LLMs can help construct test oracles for quantum programs by translating high-level intent into formal assertions and reference models despite the challenges of probabilistic behavior and limited state observability.	*The Test Oracle Problem*: Constructing reliable test oracles for quantum programs remains a fundamental challenge due to probabilistic behavior and limitations of existing validation methods, requiring more general and effective solutions for LLM-assisted testing.
Representative Works: Quantum program assertions [52, 65, 69, 73]. *Existing Challenges*: Relying on domain knowledge.	*Role*: Assisting in generating test assertions.	*The Sim-to-Real Gap*: A fundamental challenge for LLM-assisted quantum program testing is the simulation-to-reality gap, where tests effective on noiseless simulators fail to capture hardware-specific behaviors on real quantum devices.
Representative Works: Quantum program bug benchmarking [14, 101, 159, 161, 162]. *Existing Challenges*: Prone to being out of date.	*Identification of Quantum-Specific Bug Patterns*: LLMs can serve as quantum-aware static analysis tools by learning from bug benchmarks to proactively detect quantum-specific defects during development. *Roles of LLMs*: Assisting in summarizing quantum-specific bug patterns.	*Lack of High-Quality Quantum Testing Datasets*: The effectiveness of LLM-based quantum testing is constrained by the scarcity of high-quality, general-purpose datasets, creating a cycle where better models and better data are mutually dependent.
Representative Works: Debugging quantum programs [85, 89, 90, 117]. *Existing Challenges*: Unclear semantics of intermediate states.		

detection while facing challenges such as quantum-specific coverage, oracle reliability, sim-to-real gaps, and data scarcity. We discuss these aspects in the following sections.

6.1 Existing Challenges in Quantum Program Testing

Quantum program testing is one of the most active research areas in QSE [94]. A growing list of works has investigated different stages of the quantum program testing process. In this section, we categorize previous studies and discuss the challenges associated with each category.

Testing Criteria. Testing criteria define what constitutes sufficient testing by providing objective rules for designing, selecting, and terminating test cases. In the context of quantum programs, however, program representations differ significantly from those of classical software, making traditional testing criteria difficult to apply directly. To address this challenge, researchers have proposed quantum-specific testing criteria to clarify what constitutes "a good test" for quantum programs. Specifically, Ali et al. [4] and Wang et al. [135] study testing criteria based on the coverage of the inputs and outputs of the program. Long and Zhao [76] propose testing criteria derived from equivalence class partitioning, adapting a classical testing principle to quantum programs. In addition, multi-granularity coverage criteria have been explored for quantum neural networks [119]. In summary, the existing quantum program testing criteria primarily focus on input and output coverage.

Testing Techniques. To test quantum programs, researchers have explored adapting testing techniques originally developed for classical software to the quantum computing context. Various testing techniques have been investigated. Search-based testing [134, 136] has made early attempts to automatically search for test cases that more effectively satisfy predefined testing objectives. Combinatorial testing [133, 137] leverages combinations of input configurations at the qubit-level to optimize the testing process. Metamorphic testing [1, 102, 132] relies on metamorphic relations defined by domain experts to validate the output of the quantum program in the absence of explicit test oracles. Fuzz testing [131] adapts classical fuzzing techniques to generate various inputs to the quantum program. Property-based testing [50] checks the behavior of the program against formally specified properties. Concolic testing [146] combines concrete execution with constraint solving to systematically explore execution paths. Black-box testing has also been investigated for quantum programs. Long and Zhao propose methods for equivalence, identity, and unitarity checking based on observable program behavior without requiring access to the internal implementation [75]. For test input generation, Jin et al. propose NovaQ, a diversity-guided approach that uses quantum-specific features, including magnitude, phase, and entanglement, to evaluate the novelty of candidate test cases and generate diverse quantum inputs [55].

Although search- and diversity-guided approaches have been explored, defining effective optimization objectives for quantum test generation remains challenging. The probabilistic nature of quantum execution and the lack of deterministic test oracles make it difficult to evaluate the fitness of candidate test cases and guide the search toward inputs with stronger fault-revealing capability.

Assertions. Researchers have proposed various types of assertions to check the correctness of quantum program outputs. Statistical assertions [52] rely on the statistical properties obtained from repeated measurements. Projection-based assertions [65] validate the accuracy based on quantum state projections. In contrast, dynamic assertions [69, 73] focus on checking runtime behaviors under predefined conditions.

Quantum Program Bugs. Bugs in quantum programs can stem from a quantum algorithm, or the development framework and compilation kit. However, since many established quantum algorithms are supported by mathematical correctness arguments, reported bugs are more commonly found in their software implementations and supporting toolchains. Most of the reported quantum bugs are found in the implementation of quantum algorithms, such as the example shown in Figure 2, and the compilation kit, such as the bug reported in the GitHub repository.[1]

[1] https://github.com/Qiskit/qiskit/issues/7769.

```
from qiskit import QuantumCircuit,
↪  assemble
from qiskit import Aer, execute
bit = 3
bit_lst = list(range(bit))
circuit = QuantumCircuit(bit, bit)
circuit.reset(0)
circuit.reset(1)
circuit.reset(2)
circuit.x(0)
circuit.x(1)
circuit.ccx(0,1,2)
circuit.barrier()
circuit.measure(bit_lst,bit_lst)
```

(a) Buggy quantum program code because misuse of gate "CCX".

```
from qiskit import QuantumCircuit,
↪  assemble
from qiskit import Aer, execute
bit = 3
bit_lst = list(range(bit))
circuit = QuantumCircuit(bit, bit)
circuit.reset(0)
circuit.reset(1)
circuit.reset(2)
circuit.x(0)
circuit.x(1)
circuit.ccx(2,1,0)
circuit.barrier()
circuit.measure(bit_lst,bit_lst)
```

(b) Fixed quantum program code

Fig. 2. Examples of a quantum bug from Bugs4Q.

Previous studies have collected bug reports from GitHub repositories and Stack Overflow discussions to identify recurring bug patterns in quantum programs, resulting in publicly available quantum bug datasets [14, 101, 161, 162]. Based on these datasets, a human-curated bug benchmark is proposed for testing and debugging quantum programs [159]. However, the identified bugs stem mainly from the implementation attempts at quantum algorithms by developers, rather than from the algorithmic logic itself. Moreover, because of the difficulties in developing quantum programs, the number of real-world quantum bugs remains relatively small.

Debugging Techniques. Debugging complements testing by locating, diagnosing, and facilitating the repair of faults revealed by failed test cases. Miranskyy et al. [89, 90] adapt classical debugging techniques, such as brute-force, cause elimination, and backtracking, to the context of quantum programs. Metwalli and Van Meter [85] propose a debugging approach for quantum circuits that incorporates quantum-specific characteristics, including amplitude and phase information, to support debugging across different circuit types. Sato and Katsube [117] introduce an efficient bug localization method based on four key considerations for identifying faults in quantum programs. Overall, debugging quantum programs remains challenging because the semantics of intermediate quantum states are difficult to observe and interpret.

In summary, although quantum program testing has become one of the most active research areas in QSE, its challenges remain difficult to address. First, because writing quantum programs requires substantial expertise, researchers are unable to collect large-scale bug reports from developer communities, resulting in a limited availability of real test oracles. Second, many existing studies are evaluated on relatively small simulated programs, raising concerns about scalability. Finally, as quantum testing techniques are expected to be deployed on real quantum hardware, the effectiveness of existing approaches on physical quantum computers remains uncertain.

Existing Challenges of Quantum Program Testing. A major challenge in quantum program testing is the lack of reliable test oracles, as the probabilistic nature of quantum computation and the exponential state space make it difficult to determine expected outputs. Test scalability further complicates this problem, since both state-space explosion in simulation and the need for repeated executions significantly increase testing costs as program size grows. Moreover, bridging the gap

between simulator-based testing and execution on real quantum hardware remains challenging due to noise, limited qubit counts, and hardware-specific constraints.

6.2 Opportunities of LLMs for Quantum Program Testing

The application of LLMs in quantum program testing aims to simplify test configuration, identify better test cases, or even guide the testing process. Applying LLMs to testing can help address some of the current urgent challenges in the QSE ecosystem, particularly the lack of test criteria and the lack of understanding of test results.

(1) *Intelligent Generation of Quantum Test Cases*: A primary opportunity for LLMs lies in the generation of test cases and test suites. This process is fundamentally different from and more difficult than traditional software testing. In classical SE, LLMs primarily generate specific inputs and assertions to verify deterministic outputs. In QSE, the test case itself is a quantum program [153]. The LLMs must generate a valid quantum circuit to serve as a test input or a test case, a task that requires understanding of quantum programming frameworks such as Qiskit or Cirq, and the semantics of quantum gate operations. The test case generation step includes defining an initial quantum state, organizing a precise sequence of gates, and setting up the final measurement basis. Furthermore, LLMs demonstrate a remarkable capability to identify and generate tests for complex corner cases and boundary conditions that human developers might not understand [151], thus helping to generate quantum program test cases based on quantum properties.

Role of LLMs in This Task. Quantum test generation requires synthesizing valid quantum circuits as tests, not merely concrete inputs, which rule-based or template-driven methods cannot generalize [153]. LLMs can reason more effectively about quantum gate semantics, enabling the generation of legal quantum test cases. Thus, LLMs serve as necessary test-program assistants in this task.

(2) *Assistance in Test Oracle Construction*: The probabilistic nature of quantum computation makes the construction of test oracles, which is a key component for verifying the outcome of a test, a significant challenge. This difficulty is compounded by fundamental physical constraints such as the no-cloning theorem, which prevents direct inspection of an intermediate quantum state and complicates debugging. LLMs offer a promising approach to alleviate this problem. By parsing high-level algorithm descriptions or developer comments, LLMs can offer suggestions of classical algorithms or mathematical models to serve as a reference oracle. Additionally, LLMs can implement the developer's intention into formal, machine-verifiable assertions. This aligns with existing QSE research on quantum-specific assertions, where an LLM could automatically generate the required assertion code to verify properties such as state fidelity or expected measurement probabilities [52].

Role of LLMs in This Task. Constructing test oracles for quantum programs is challenging due to probabilistic outputs and physical constraints such as the no-cloning theorem, which limit direct state inspection [52]. LLMs can infer reference models and help generate quantum assertions, acting as a necessary semantic oracle synthesizer in this task.

(3) *Identification of Quantum-Specific Bug Patterns*: Bugs in quantum programs are not limited to the logical errors found in classical code; they also include subtle and non-intuitive errors that violate quantum mechanical principles [101]. By fine-tuning benchmarks that contain a corpus of known real-world quantum program bugs, such as Bugs4Q [159], LLMs can learn to identify the features of these quantum-specific bug patterns. During development, an LLM can act as a proactive, quantum-aware static analysis tool that scans code to alert developers

to potential risks, such as uninitialized qubits or incorrect gate sequences, thus enabling earlier bug detection in the development cycle.

Role of LLMs in This Task. Quantum bugs include violations of quantum-mechanical principles that evade classical static analyses [101]. Related benchmark studies show that such bugs typically appear in diverse forms [159]. Hence, LLMs are necessary quantum-aware pattern learners.

6.3 Key Research Challenges

Although LLMs show great potential, their effective and reliable application to quantum program testing requires overcoming key challenges arising from the inherent properties of quantum computation.

6.3.1 Challenge 1: Quantum-Specific Test Coverage. Traditional test coverage criteria, such as statement and branch coverage, cannot be directly adopted for quantum programs. This is because the core logic of a quantum program lies in the continuous, unitary computation of its quantum state, not in a discrete, classical control flow. For example, the coverage criterion of magnitude score, phase score, and entanglement score is proposed by Ye et al. [153]. Consequently, conventional testing approaches are inadequate to cover the vast state space of a quantum program [153]. Although new quantum-specific coverage criteria are being proposed, such as input-output coverage [4] and gate branch coverage [36], they are still in the exploratory stage and remain a large improvement space to fully capture the characteristics of quantum states. The challenge for LLMs is thus twofold: first, to understand quantum properties and program features, and second, to construct a minimal set of test cases that maximizes state-space coverage by integrating quantum-specific features.

6.3.2 Challenge 2: The Test Oracle Problem. Providing a precise oracle for a quantum program is rather difficult. Due to the probabilistic nature of quantum programs, building an oracle for the testing outcome usually faces the incompleteness challenge. This becomes even more difficult for algorithms designed for quantum purposes. To alleviate this, researchers are exploring techniques such as metamorphic testing for quantum programs [1], which establish relations among properties of the program across different inputs and outputs (e.g., applying a gate followed by its inverse should be an identity operation). Black-box testing provides another way to address the oracle problem by checking properties such as program equivalence, identity, and unitarity from observable input–output behavior [75]. However, the metamorphic testing approach does not work for all cases in testing quantum programs because it is difficult to write relations in many cases. Moreover, in recent studies [87, 152], the testing result validation approach is discussed. Researchers report that existing sampling-measurement-based validation approaches may be insufficient for current quantum program testing, and propose using statevector-based validation as a complement to existing approaches. Therefore, considering general support for quantum program testing, a challenge for LLMs' application should be the test oracle problem.

6.3.3 Challenge 3: The Sim-to-Real Gap. Most existing quantum program testing techniques, including recent approaches assisted by LLMs, are still predominantly conducted on ideal, noiseless simulators. While simulators provide full controllability and observability, the execution behavior of quantum programs on real **Noisy Intermediate-Scale Quantum (NISQ)** hardware can differ substantially due to physical effects such as gate infidelity, qubit decoherence, crosstalk, and measurement errors. These hardware-specific factors are difficult to accurately model or anticipate in simulation-based testing workflows. As a result, even when LLMs are applied to assist test generation, analysis, or prioritization, the generated tests may fail to expose behaviors that only manifest on real devices. This leads to a persistent simulation-to-reality gap, where test effectiveness

observed on simulators does not reliably scale to a real quantum machine. In this regard, Muqeet et al. [93] propose a machine learning-based approach to learn the characteristics of noise in quantum program outputs and subsequently filter it according to the detected noise patterns. While this represents an important early attempt toward addressing the sim-to-real gap in quantum program execution, the approach remains preliminary and leaves substantial room for further improvement. Consequently, the sim-to-real difference remains a fundamental challenge for LLM-assisted quantum program testing and must be considered when evaluating testing effectiveness and practical applicability.

6.3.4 Challenge 4: Lack of High-Quality Quantum Testing Datasets. A fundamental challenge for any data-driven approach is the availability of training data. For QSE, there is a notable lack of high-quality datasets of quantum programs for training and evaluation. Existing benchmarks are often either synthetic datasets that lack clear explanations [17] or tailored to a very specific use case [64]. Other benchmarks such as Bugs4Q [159], VQE [95], Nation et al. [97], MQT [113], and MNISQ [111] serve specific tasks, which are not well generalized to all testing tasks in quantum programs. This scarcity hinders the ability to fine-tune LLMs to understand the specific differences of quantum testing. It creates a challenge: improving LLM-based tools requires better data, but creating reliable, well-tested datasets of quantum programs is difficult without better LLMs.

Challenges of Applying LLMs in Quantum Software Testing

Challenge 1: Quantum-specific test coverage
Challenge 2: The test oracle problem
Challenge 3: Simulation-to-real gap
Challenge 4: Lack of high-quality quantum testing dataset

7 LLMs for Repairing Quantum Programs

7.1 Current State and Opportunities

Program repair is critical to improving program reliability. Currently, LLMs have created new possibilities for automating quantum program repair. In a recent survey, LLMs have been applied to improve classical SE approaches, including the analysis of runtime logs, error messages, and bug reports to identify and locate faults [157]. By analyzing characteristic error patterns and linking them to concrete faults in quantum gate operations or qubit handling, LLMs can propose targeted fixes and, in many cases, automatically generate valid repair patches.

Table 4 summarizes representative work, key challenges, and emerging opportunities in quantum program repair, highlighting how LLMs enable automated patch generation and workflow coordination while facing challenges such as semantic gaps, patch correctness, data scarcity, and hardware-aware repair constraints.

A previous study by Guo et al. [47] demonstrated that LLMs such as ChatGPT can successfully repair a large proportion of the bugs in Bugs4Q [159], which is a benchmark of real-world quantum program defects. Their work revealed a crucial insight: the richness and quality of the context provided to the LLM, such as precise fault locations or hints about the bug's nature, can dramatically improve the success rate of the repair process. This suggests that LLMs are already capable of serving as powerful assistants to developers, automating the process of generating potential fixes based on diagnostic information. Such capability can contribute to the long-term maintenance of a quantum program dataset by performing code refactoring, thereby enhancing the overall quality of the whole repository.

Table 4. Existing Challenges in Quantum Program Repair and Future Opportunities by Adopting LLMs

Representative Works and Existing Challenges	Opportunities and Roles of LLMs	Future Challenges
Representative Works: Quantum program repair using unitary operation [68]. *Existing challenges*: Limited scalability and reliance on repeated measurement.	*Automated Patch Generation and Repair*: LLMs can analyze error patterns, runtime logs, and bug reports to generate targeted repair patches, improving repair automation. *Roles of LLMs*: Assisting patch generation and automated repair.	*Understanding Quantum-Specific Errors*: A key challenge is the semantic gap, where LLMs lack understanding of quantum-specific errors, leading to syntactically correct but semantically incorrect repairs.
	Coordinating End-to-end Repair Workflow: LLMs can integrate fault localization, patch generation, and validation into a unified repair pipeline through multi-step reasoning. *Roles of LLMs*: Acting as repair coordinators and reasoning agents.	*The Patch Correctness Problem*: Verifying repairs is challenging because correctness must be assessed statistically over probabilistic outputs, making validation computationally expensive.
Representative Works: LLM-based quantum program repair [47]. *Existing Challenges*: Dependence on high-quality contextual information.		*Scarcity of High-Quality Training Data*: The effectiveness is fundamentally limited by the scarcity of large-scale, high-quality, and well-annotated datasets of quantum bugs and fixes.
	Hardware-Aware Repair Assistance: LLMs can incorporate hardware information such as topology and noise models to guide repair decisions. *Roles of LLMs*: Assisting hardware-aware repair.	*Hardware-Aware Repair and the Simulation-to-Hardware Gap*: A key challenge is the simulation-to-hardware gap, where patches valid in noiseless simulators may fail on real devices due to hardware-specific noise and constraints.

Role of LLMs in This Task. Repairing quantum programs is inherently complex, typically involving multiple stages, including program analysis, patch generation, and repair validation. Integrating these stages into a coherent and effective repair pipeline remains challenging for classical automated approaches, which often address each step in isolation and lack the ability to coordinate reasoning across stages. In contrast, recent studies demonstrate that LLMs excel at orchestrating multi-step SE workflows by jointly reasoning over program structure, repair intent, and validation feedback [149]. Such capabilities are particularly valuable in the quantum program context, where repair requires simultaneous consideration of program semantics, quantum constraints, and testing limitations. These advantages position LLMs as essential coordinators and advisors capable of managing and integrating the end-to-end quantum program repair process.

7.2 Existing Challenges in Quantum Program Repair

Repairing quantum programs is particularly challenging. The **Automatic Program Repair (APR)** paradigm typically consists of three stages: fault localization, patch generation, and repair validation. In the context of quantum programs, each of these stages faces substantial difficulties. For example, fault localization is difficult because the semantics of intermediate states in a quantum circuit are difficult to observe and interpret. Only a limited number of studies have investigated bug fixing and repair for quantum programs. Luo et al. conduct an empirical study of 96 real-world bugs and their fixes across Qiskit, Cirq, Q#, and ProjectQ, showing that more than 80% of the collected bugs are quantum-specific and identifying common bug patterns [80]. Existing automated repair studies mainly include UnitAR proposed by Li et al. [68] and the LLM-based repair approach proposed by Guo et al. [47]. The following paragraphs discuss these studies in detail.

Li et al. [68] propose UnitAR, an approach to repair quantum programs by modifying unitary operations. Their method first executes the quantum program and then collects the output probability distributions via repeated sampling. By comparing the observed output probabilities with the expected ones, UnitAR generates patches that insert additional unitary operations at intermediate positions in the quantum circuit, thereby adjusting the final output distribution. Although this approach is limited by measurement overhead and limited scalability in testing, it offers a valuable insight: repairing a quantum program can be considered as repairing the quantum state transformations induced by the program.

Guo et al. [47] investigate the application of LLMs to quantum program repair. Their study investigates the extent to which LLMs can effectively assist in repairing quantum programs. To this end, they design prompts that integrate multiple levels of program information to determine the amount and granularity of contextual information required to generate effective repair patches. In their evaluation, LLMs are applied to fix bugs from the Bugs4Q benchmark, successfully fixing 29 out of 38 bugs. Overall, this work demonstrates the potential of LLM-assisted approaches as a promising paradigm for QSE.

In summary, repairing quantum programs is inherently challenging because the repair process inherits the difficulties of both quantum program generation and testing. First, generating repair patches for quantum programs is non-trivial, as it faces challenges similar to those encountered in quantum program generation, such as complex program semantics, strict syntactic constraints, and the need to preserve quantum state evolution. Second, bug localization and repair validation are equally difficult, as they share the same limitations as quantum program testing, including the oracle problem, limited observability of intermediate states, and scalability constraints in simulation-based evaluation. Despite these challenges, existing studies demonstrate promising opportunities. Li et al. [68] showed that repairing quantum programs can be reformulated as repairing quantum state transformations, providing a novel state-centric perspective. Guo et al. [47] further illustrate the practical feasibility of quantum program repair by applying LLMs to automatically fix real bugs in an open benchmark. Together, these works suggest that quantum program repair, while challenging, is a viable and emerging research direction.

Existing Challenges of Quantum Program Repair. A primary challenge in quantum program repair is accurately locating bugs, as errors may arise from subtle interactions between quantum operations, measurement outcomes, and hardware constraints. Even when faulty regions can be identified, generating effective patches is difficult because modifications must preserve quantum semantics and avoid introducing new errors. Furthermore, validating generated patches remains challenging due to the lack of reliable oracles and the probabilistic behavior of quantum program executions.

7.3 Challenges and Future Directions

Despite these promising initial results, applying LLMs to quantum program repair presents a set of challenges that are fundamentally different and more complex than those in the classical domain. These differences stem from the inherent nature of quantum mechanics and the current state of the quantum software ecosystem. Future research should focus on the following key areas.

7.3.1 Challenge 1: Understanding Quantum-Specific Errors. A primary challenge lies in the semantic gap: LLMs are trained in classical code and natural language, leaving them largely unaware of bugs in quantum programs, such as the misuse of superposition, entanglement, and state collapse [101]. While classical bugs, such as null pointer exceptions or off-by-one errors, typically follow logical patterns, quantum programs exhibit unique bug categories that are often non-intuitive for models pre-trained on classical datasets. These include:

(1) *Incorrect Gate Implementation*: Using the wrong quantum gate or applying it to the wrong qubits.
(2) *State Preparation and Measurement Errors*: Failing to initialize qubits to the correct state or misinterpreting measurement outcomes.
(3) *Quantum-Specific Logic Errors*: Flaws in the implementation of a quantum algorithm, such as incorrect gate manipulation in Grover's algorithm or flawed oracle design [162].

Without an in-depth semantic understanding of these error types, LLMs risk generating syntactically correct but semantically incorrect patches. Such patches might resolve a compilation error, but completely alter the program's quantum behavior. These unexpectedly introduced changes in the quantum program are much harder to detect than those in classical programs. Future work should focus on developing novel techniques, such as pre-training on quantum physics literature or integrating knowledge graphs, to equip LLMs with a quantum-specific knowledge network.

7.3.2 Challenge 2: The Patch Correctness Problem. Verifying the correctness of a generated patch is a well-recognized challenge in classical APR. In the quantum setting, however, this challenge becomes substantially more difficult. Due to the probabilistic nature of quantum computation, a program does not yield a single deterministic output, but instead produces samples drawn from an underlying probability distribution over measurement outcomes. This means that validating a patched quantum program requires reasoning not about individual executions, but about whether the resulting output distribution aligns with the theoretically expected one.

Unlike classical software, where a well-designed test suite can often provide near-definitive evidence of correctness, correctness validation in quantum program repair inevitably becomes a statistical inference task. A patched program must typically be executed repeatedly, often thousands of times, to estimate its output distribution, and this distribution must then be statistically compared against a reference distribution. Existing work on statistical assertions demonstrates the importance of statistical hypothesis testing when assessing quantum program correctness [52]. Furthermore, recent advances in quantum equivalence checking and conformance validation indicate that formal comparison between the repaired program and a specification circuit remains challenging, especially in the presence of realistic noise, device variability, and NISQ constraints [126]. These factors make validation both computationally expensive and methodologically complex.

7.3.3 Challenge 3: Scarcity of High-Quality Training Data. The effectiveness of LLM-based repair techniques fundamentally depends on the availability of large, representative, and well-annotated training data [82, 148]. In classical SE, such data is abundant: decades of open source repositories, bug reports, version histories, and patch corpora provide rich supervision for training and evaluating repair systems [144]. In contrast, the quantum software ecosystem remains comparatively young.

Real-world quantum applications are fewer, mature industrial-scale projects are rare, and high-quality datasets containing buggy quantum programs paired with human-crafted fixes are extremely limited. Although empirical studies of real-world quantum bugs and their fixes [80], as well as early benchmarks such as Bugs4Q [159] and QBugs [14], provide valuable initial resources, the available data remain limited in scale, diversity, semantic richness, and coverage of realistic fault scenarios.

This scarcity presents a fundamental bottleneck for developing LLMs capable of reliably repairing quantum programs. General-purpose models trained predominantly on classical code are unlikely to capture quantum-specific algorithmic patterns, hardware constraints, noise behavior, and semantic subtleties, such as entanglement handling and state-preparation correctness [61]. Without sufficient quantum-domain supervision, LLMs risk producing superficially valid but semantically incorrect repairs or overfitting to the narrow characteristics of benchmarks.

Addressing this challenge requires a coordinated community effort to construct large-scale, diverse, and semantically annotated datasets of quantum bugs and patches [127]. Beyond manually curated benchmarks, future work should explore automated mining of quantum repositories, fault injection frameworks that mimic realistic quantum defects, synthetic-but-semantic bug generation, and the collection of real-world failure cases from both simulators and hardware executions. Such datasets will be essential for enabling domain-specialized fine-tuning, rigorous evaluation, and trustworthy LLM-driven repair of quantum programs.

7.3.4 Challenge 4: Hardware-Aware Repair and the Simulation-to-Hardware Gap. A critical challenge is the gap between idealized quantum simulators and noisy real-world quantum hardware. A patch that works perfectly in a noiseless simulation may fail on a physical Quantum Processing Unit due to hardware-specific constraints and errors, such as:

(1) *Decoherence*: The loss of quantum information as a result of interaction with the environment.
(2) *Gate Fidelity Errors*: Quantum gates that do not perform their intended operation perfectly.
(3) *Qubit Connectivity Limitations*: Not all qubits on a chip are physically connected, requiring complex "SWAP" operations that can introduce additional errors.

An effective quantum program repair tool cannot be hardware-agnostic. The generated patch must be robust and efficient when executed on a specific target device. This requires developing "hardware-aware" LLMs for program repair. Such models would need to take as input not only the buggy code but also a description of the target hardware's topology, noise model, and gate set, thereby enabling them to generate patches optimized for real-world execution.

Challenges in Quantum Software Repair with LLMs

Challenge 1: Semantic gap and understanding quantum-specific errors
Challenge 2: The patch correctness problem
Challenge 3: Lack of high-quality quantum repair dataset
Challenge 4: Hardware-aware repair and simulation-to-hardware gap

8 Future Research

8.1 Summary of Challenges

Table 5 categorizes key challenges in LLM-based QSE across modeling, generation, testing, and repair, highlighting recurring bottlenecks such as data scarcity, semantic correctness, and the simulation-to-real gap. These challenges evolve from fundamental issues in the short-term to

Table 5. Taxonomy of Challenges and Bottlenecks in LLM-Based QSE

ID	Specific Challenges	Challenge Taxonomy	Key Bottlenecks	Position in Timeline
Task A: Quantum Software Modeling and Analysis				
C1	Hallucination of Analysis	Practicality/Scalability	Quantum Code Safety and Robustness	Mid-term
C2	Scalability of Program Analysis	Practicality/Scalability	Simulation-to-Real Gap	Mid-term
C3	Lack of Labeled Analysis Data	Fundamental Issues	Data Scarcity	Short-term
Task B: Generating Quantum Software				
C4	Program Correctness Gap	Fundamental Issues	Semantic Correctness and Quantum Errors	Short-Term
C5	Quantum-Specific Space Exploration	Practicality/Scalability	Physical Constraints and Hardware-aware Repair	Mid-Term
C6	Lack of High-Quality Training Data	Fundamental Issues	Data Scarcity	Short-Term
C7	Reducing Software Failures	Practicality/Scalability	Quantum Code Safety and Robustness	Mid-Term
C8	Human-Centric Innovation Balance	Human–AI Collaboration	Automation vs. Innovation	Long-Term
Task C: Testing Quantum Software				
C9	Quantum-Specific Test Coverage	Human–AI Collaboration	Testing Complex Quantum Properties	Long-Term
C10	The Test Oracle Problem	Fundamental Issues	Test Oracle Reliability	Short-Term
C11	Simulation-to-Real Gap	Practicality/Scalability	Simulation-to-Real Gap	Mid-Term
C12	Lack of Quantum Testing Dataset	Fundamental Issues	Data Scarcity	Short-Term
Task D: Repairing Quantum Software				
C13	Semantic Gap and Error Understanding	Fundamental Issues	Semantic Correctness and Quantum Errors	Short-Term
C14	The Patch Correctness Problem	Fundamental Issues	Test Oracle Reliability	Short-Term
C15	Hardware-Aware Repair	Practicality/Scalability	Hardware-Aware Repair	Mid-Term
C16	Lack of Quantum Repair Dataset	Fundamental Issues	Data Scarcity	Short-Term

scalability concerns in the mid-term and human–AI collaboration problems in the long-term. The listed challenges and opportunities are elaborated in detail in the following sections.

8.2 Future Timeline and Research Opportunities

As summarized in Table 5, the scarcity of high-quality training data is a key challenge that recurs across generation, testing, and repair, making it the most pressing issue to address. Similarly, the semantic gap, the LLM's lack of inherent understanding of quantum principles, is identified as another fundamental obstacle. In short, some of the challenges outlined above are urgent and should be addressed in the near term, whereas others can wait until the QSE ecosystems mature, allowing for more effective solutions. Based on these findings, we categorize the challenges into a timeline of three phases: short-term, mid-term, and long-term goals.

Figure 3 presents a timeline for how LLMs will progressively change QSE from 2025 to 2030, highlighting key challenges and opportunities in short-, medium-, and long-term stages, moving

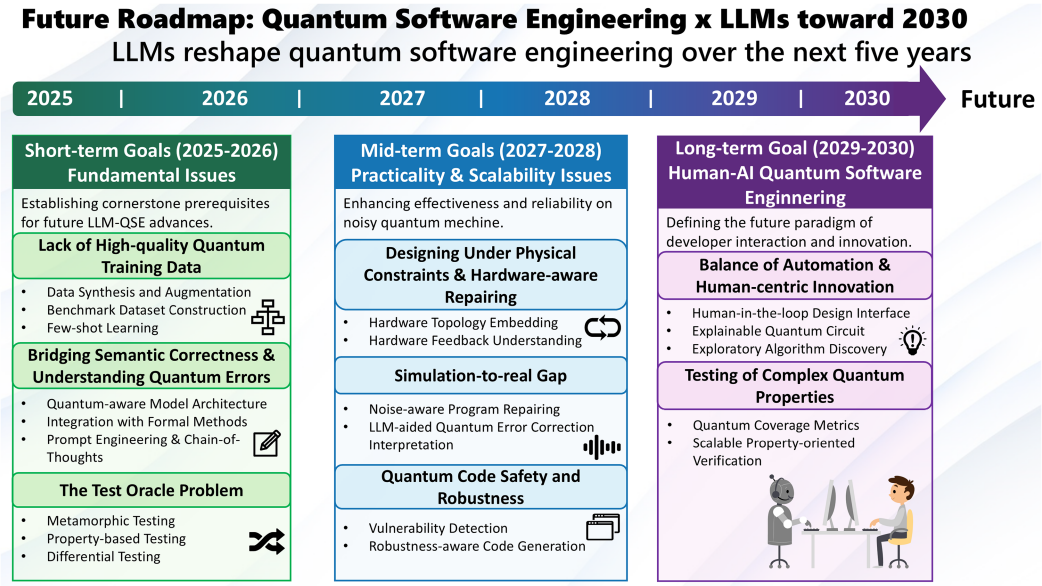


Fig. 3. A timeline roadmap of LLMs in QSE toward 2030.

from foundational issues to scalable systems and human–AI collaboration. Each challenge and opportunity is discussed in the following sections.

8.2.1 Short-Term Goals: Fundamental Issues. The short-term focus should be on fundamental issues whose resolution is a prerequisite for future advances. Addressing these issues will establish the foundation for future research and application in LLM-powered QSE.

(1) *Lack of High-Quality Quantum Training Data* (Modeling and Analysis Challenge 3, Generation Challenge 3, Testing Challenge 4, Repair Challenge 4)
   — *Importance*: The lack of diverse, high-quality training datasets remains the primary obstacle to the maturation of LLM-driven QSE. The performance of the LLMs is directly dependent on the quality and volume of available training data. Without sufficient, high-quality examples of quantum programs, test suites, and bug-fix commits, LLMs cannot learn the quantum-specific patterns required for effective code generation, comprehensive testing, or reliable program repair.
   — *Research Opportunities*:
      – *Data Synthesis and Augmentation.* Existing work in classical SE has demonstrated that synthetic and augmented data can significantly improve LLM performance, especially when real-world data is scarce [6, 27, 70, 156]. For quantum software, this opens several concrete research directions. One is to develop quantum program synthesis pipelines that automatically generate diverse, structurally rich program samples that cover a variety of algorithmic patterns, circuit depths, qubit scales, and noise models. Another direction is to design task-aware augmentation strategies, such as generating equivalent but syntactically different programs, adding controlled perturbations, or simulating buggy code variants. Since synthetic data inevitably contains noise and code smells, an important complementary problem is to design automatic filtering, quality assessment,

and dataset-refinement mechanisms to ensure that only semantically meaningful and representative samples are used for LLM training.

– *Benchmark Dataset Construction.* The lack of unified, high-quality benchmarks significantly limits fair comparison, reproducibility, and meaningful progress in QSE. Existing benchmarks such as QASMBench [64], Bugs4Q [159], QBugs [14], VQE [95], Nation et al. [97], MQT [113], and MNISQ [111] are limited in scalability. Therefore, a research opportunity is to design a comprehensive quantum software benchmark suite that serves program generation, testing, debugging, compilation, and repair tasks. Such a benchmark should include representative real-world quantum applications, scalable synthetic workloads, standardized evaluation protocols, labeled vulnerabilities, and well-curated execution traces. Beyond static datasets, dynamic benchmarking environments that integrate simulators and hardware backends may also be required to capture the behavior of real quantum programs during execution.

– *Few-Shot Learning*. Given that quantum software data will likely remain scarce for the near future, improving data efficiency is as important as increasing dataset volume. Few-shot learning offers a promising approach by enabling models to generalize from a very few examples [11, 124]. Future work may explore how prompt engineering, instruction tuning, RAG, or meta-learning techniques can improve few-shot capability in quantum domains. Another promising direction is to combine few-shot learning with domain knowledge, such as physics constraints, compiler semantics, or execution traces, so that models rely less on raw data volume and more on structured quantum understanding. These approaches can lower the data requirement barrier, making LLM-assisted QSE feasible even in realistically data-limited settings.

(2) *Bridging the Gap of Semantic Correctness and Understanding the Quantum Errors* (Generation Challenge 1, Repair Challenge 1)

— *Importance*: The semantic gap is another fundamental obstacle. An LLM that fails to comprehend quantum principles like superposition and entanglement is likely to generate syntactically correct but semantically flawed code that fails on both simulators and quantum hardware. Furthermore, without a deep understanding of a program's intended quantum behavior, an LLM cannot effectively generate meaningful test cases or propose correct repairs.

— *Research Opportunities*:

– *Quantum-Aware Model Architecture*. A key direction is to move beyond treating quantum code as plain text and instead design model architectures that explicitly encode quantum semantics. This may involve incorporating representations of quantum states, unitary evolution, entanglement structure, or circuit depth constraints into the pre-training or adaptation process. For example, hybrid learning frameworks could combine symbolic reasoning over circuit structures with neural representations, allowing the model to learn not only syntactic patterns but also physically meaningful relationships [60]. Another complementary direction is to embed quantum attributes, such as basis encoding, amplitude encoding, and angle encoding, so that it facilitates researchers and developers in developing smart quantum algorithms [59].

– *Integration with Formal Method.* Formal methods can be leveraged to provide a precise semantic specification of quantum programs under ideal execution assumptions, serving as a reference model for semantic correctness. When combined with LLMs, this formal semantic baseline can be contrasted with execution behaviors observed on noisy quantum hardware [35]. In this setting, LLMs can assist in interpreting differences between formally verified semantics and execution outcomes, helping to distinguish semantic defects from

quantum errors and improving understanding of how quantum errors manifest at the level of program semantics [86].

– *Prompt Engineering and Chain-of-Thought.* Even without fundamentally changing model architecture, substantial gains may be achievable by guiding how LLMs reason about quantum programs. One promising direction is the development of structured prompting strategies that explicitly encourage step-by-step semantic reasoning, such as requiring the model to analyze circuit intent, expected state evolution, and potential semantic pitfalls before producing final code [38]. Chain-of-thought prompting can be tailored to approximate expert reasoning workflows by exposing intermediate reasoning steps and incorporating step-level self-verification [72, 140]. In QSE, such prompts can further guide the model to perform incremental correctness checks, track relevant invariants, and validate its reasoning against established quantum principles [15]. Prompt templates, reasoning scaffolds, and role-based prompting may together help transform LLMs from passive code generators into more reflective and semantically aware quantum programming assistants.

(3) *The Test Oracle Problem* (Testing Challenge 2, Repair Challenge 2)

— *Importance*: The test oracle problem, a longstanding challenge in SE, becomes significantly more severe in the quantum domain. Due to the inherent feature of quantum computing, a program's state collapses upon measurement. Therefore, unless repeating well-known quantum algorithms, it is difficult to determine the correct output of an arbitrary quantum program. This inherent uncertainty makes it difficult to verify test outcomes and patch correctness. And it will be challenging to figure out whether a test has passed or a repair is valid.

— *Research Opportunities*:

– *Metamorphic Testing*. While metamorphic testing has been widely adopted to alleviate the oracle problem, its effectiveness in the quantum domain is fundamentally limited by the difficulty of identifying meaningful metamorphic relations for arbitrary quantum programs [102]. Existing approaches typically rely on manually designed relations or domain-specific knowledge, which do not scale to diverse quantum algorithms. LLMs can assist by learning and hypothesizing semantically meaningful input–output relations from existing quantum programs and test histories, enabling the automated discovery of metamorphic relations beyond heuristic rules.

– *Property-Based Testing*. Property-based testing requires explicit specifications or assertions describing expected program behaviors, which are particularly hard to formalize for probabilistic quantum outputs. In practice, many quantum programs lack well-defined, human-writable properties beyond a few classical algorithms [52]. LLMs offer a promising approach for inferring high-level behavioral properties from program structure, documentation, and prior executions, thereby reducing reliance on manually written assertions and improving the applicability of property-based testing in quantum software.

– *Differential Testing*. Differential testing mitigates the oracle problem by comparing the behaviors of semantically equivalent implementations, and has been applied in testing quantum frameworks [132]. However, constructing diverse yet semantically equivalent quantum program variants remains a major challenge, especially in the presence of complex compiler optimizations and hardware constraints. LLMs can contribute by generating semantically equivalent but syntactically diverse program variants at scale, expanding the space of differential comparisons and enabling more effective detection of subtle inconsistencies.

8.2.2 Mid-Term Goals: Practicality and Scalability Issues. After addressing the correctness and the lack of training data, research should focus on enhancing the effectiveness and reliability of LLM-generated quantum programs when executed on real, noisy quantum computing systems. Success in this phase can help establish scalable, hardware-aware, and robust quantum software systems.

(1) *Designing under Physical Constraints and Hardware-Aware Repair* (Generation Challenge 2, Repair Challenge 3)
   — *Importance*: With the advancement of NISQ hardware, it is difficult to discuss the software without considering the effect of quantum hardware. A critical mid-term goal is therefore to enable LLMs to understand and adapt to physical hardware constraints, such as qubit connectivity, decoherence times, and gate error rates.
   — *Research Opportunities*:
      – *Hardware Topology Embedding*. A promising direction is to explicitly expose hardware constraints and execution feedback to LLMs during quantum program generation or repair, rather than assuming hardware-agnostic coding [86]. This includes embedding device-specific coupling maps, qubit connectivity graphs, calibration data, noise models, and gate duration constraints into the model's input representation or training pipeline. With such knowledge, LLMs could automatically select hardware-feasible gate decompositions, choose qubits with lower error rates for critical operations, or generate routing strategies that minimize SWAP insertion. Beyond static embedding, research is needed to determine how frequently topology and calibration information should be refreshed, how to encode temporal fluctuations in hardware quality, and how to balance between portability and device optimization.
      – *Hardware Feedback*. Another important opportunity lies in creating closed-loop frameworks where LLMs iteratively interact with real or simulated quantum hardware. In such a setting, LLMs would generate candidate circuits or repairs, execute them on hardware or realistic simulators, and analyze execution outcomes, including fidelity, error rates, and performance variance. These results can then be fed back into the model to enhance beneficial design choices and penalize ineffective ones [13]. This raises research challenges such as designing effective reward functions, integrating reinforcement learning or online adaptation, and distinguishing hardware noise effects from semantically incorrect behavior. Further, such feedback-driven processes could enable LLMs to evolve from static code generators into adaptive agents that progressively optimize circuits for specific hardware backends while maintaining correctness and robustness.

(2) *Simulation-to-Real Gap* (Modeling and Analysis Challenge 2, Testing Challenge 3, Repair Challenge 3)
   — *Importance*: A program that executes perfectly on a simulator may still fail on a real quantum device due to hardware noise. Bridging this simulation-to-reality gap is a critical step towards developing robust and reliable quantum software.
   — *Research Opportunities*:
      – *Noise-Aware Program Repair*. In simulation-to-real testing, the core challenge is not to explicitly model hardware noise, but to determine which program behaviors are expected to remain semantically stable under real noise conditions. Traditional machine learning techniques typically rely on labeled data or explicit noise features, which are often unavailable, device-dependent, or unstable over time [93]. In contrast, by fine-tuning with a certain size of quantum circuits, LLMs can effectively generate syntactically correct parametrized quantum circuits [54].

– *LLM-Aided Quantum Error Correction.* Quantum error correction and error mitigation are performed at the hardware or runtime level and cannot be controlled by quantum program developers. However, they directly affect the execution results observed during testing and repair, making it difficult to determine whether an unexpected outcome is due to a program defect or hardware noise. Recent work has begun to explore the use of LLMs in quantum error correction. For example, Campbell et al. [13] propose a multi-agent LLM framework for generating fault-tolerant quantum programs, in which specialized agents iteratively refine quantum code through semantic analysis and interaction with an error-correction code decoder. By reasoning over program semantics, compiler behavior, and execution patterns, LLMs can support more reliable repair validation without crossing into noise tolerance.

(3) *Quantum Code Safety and Robustness* (Modeling and Analysis Challenge 1, Generation Challenge 4)
— *Importance*: Once LLMs can generate the correct quantum program code, discussions on code safety and robustness become crucial. This includes preventing potential vulnerabilities in quantum algorithms and ensuring that the program behavior is consistent under all circumstances.
— *Research Opportunities*:
– *Quantum Program Vulnerability Detection.* Existing work has shown that formalizing QIRs can provide a rigorous foundation for detecting unsafe operations and improving code safety [79]. Rather than replacing such formal and static-analysis techniques, LLMs can be used to complement them by identifying potential safety and robustness issues that are difficult to capture with fixed rules or predefined patterns [121]. In particular, LLMs are well-suited to reason about high-level program intent and semantic consistency, which is increasingly important when quantum programs are automatically generated and may exhibit unconventional structures or implicit assumptions that violate robustness expectations.
– *Robustness-Aware Code Generation.* Rather than introducing heavyweight adversarial training schemes, robustness can be improved by incorporating lightweight feedback into the code generation process. For example, issues identified by static analysis, testing, or prior executions can be summarized and used to guide subsequent generations [121]. In this way, LLMs do not act as a robustness-verification tool but as an assistant that adapts their output in response to observed weaknesses, thus gradually reducing the likelihood of producing fragile or unsafe quantum programs.

8.2.3 Long-Term Goals: Human–AI Collaboration and Vision. These long-term goals address broader, forward-looking challenges that will define the future paradigm of quantum software development. They can only be systematically tackled once the underlying LLM-for-QSE technology has reached a high level of maturity. The achievement of long-term goals enables human–AI collaboration and drives innovation in QSE.

(1) *Balance of Automation and Human-Centric Innovation* (Generation Challenge 5)
— *Importance*: This goal addresses the ultimate question governing the human–AI relationship in QSE: As LLMs become more powerful, should they act as black-box automation tools that replace developers or as intelligent partners that empower human creativity? Answering this question involves fundamental considerations of technology, ethics, and development workflows and will require sustained long-term exploration.

— *Research Opportunities*:
  – *Human-in-the-Loop Design Interface*. An important research direction is the design of interactive development environments in which LLMs generate multiple candidate quantum circuits or program variants, accompanied by structured explanations of their design rationales, tradeoffs, and expected behaviors [13]. Human experts can then evaluate, refine, and steer these candidates based on domain knowledge and experimental goals. Such interfaces emphasize collaboration rather than replacement, enabling humans to remain in control while benefiting from the capacity of LLMs.
  – *Explainable AI*. As quantum programs grow in complexity, understanding why a particular algorithm structure or repair decision is generated becomes increasingly important. Prior work on LLM-driven debugging shows that exposing the hypotheses and reasoning behind generated patches can help developers assess patch correctness [57]. Future research should investigate how LLMs can explain quantum program outputs and repair decisions by identifying key design choices, assumptions, and semantic constraints. Such explanations can support trust and verification and may also facilitate knowledge transfer from model-generated solutions to human developers.
  – *Exploratory Algorithm Discovery*. Instead of relying on existing patterns, LLMs present an opportunity to support the exploratory discovery of novel quantum algorithm structures. By synthesizing ideas from existing algorithms, theoretical principles, and experimental feedback, LLMs can help generate unconventional circuit designs that may inspire new research directions. In this role, LLMs act as a creative aid, expanding the search space of algorithmic possibilities while leaving validation, interpretation, and theoretical grounding to human researchers [32].

(2) *Testing of Complex Quantum Properties* (Testing Challenge 1)
— *Importance*: Although this problem is fundamental, its scope and complexity make it a long-term challenge. Achieving comprehensive coverage and verification of complex quantum properties, including entanglement and superposition, remains exceptionally difficult. Substantial research effort and theoretical breakthroughs in QSE are required.
— *Research Opportunities*:
  – *Quantum Coverage Metrics*. Existing coverage metrics from classical software testing cannot be directly applied to quantum programs, as quantum states and correlations cannot be exhaustively enumerated or directly observed [119]. An open research opportunity is to develop coverage metrics that approximate the exploration of quantum behaviors, such as state magnitudes, phases, entanglement structures, and measurement outcome distributions, and to use these metrics to guide the generation of diverse quantum test inputs [153]. Building on this direction, future work could investigate how LLMs can generate or refine test inputs to systematically exercise quantum behaviors that remain insufficiently covered.
  – *Scalable Property-Oriented Verification*. Verifying complex quantum properties at scale becomes increasingly difficult as the number of qubits grows, due to the exponential state space and limited observability. Rather than full-state verification, research is needed to explore property-oriented testing approaches that focus on partial, high-level behavioral properties. This opens opportunities to study how LLMs, simulators, and verification tools can be combined to reason about and validate such properties without requiring exhaustive simulation [112].

9 Conclusion

In this article, we have established promising cases for integrating LLMs into QSE practices and presented an initial roadmap to guide future research efforts in this emerging field. In the roadmap, we summarize four urgently required areas of research. First, we discuss the potential of LLMs in quantum program modeling and analysis. Second, we highlight the role of LLMs in enhancing quantum program code generation methodologies. Third, we discuss the promising domain of LLM-assisted quantum program testing. Finally, we summarize the challenges and opportunities in quantum program repair with LLMs. Although these four directions represent significant research opportunities, we acknowledge that they do not constitute an exhaustive representation of the potential applications of LLMs in QSE. We invite our fellow researchers to contribute to this emerging field by identifying additional research gaps and challenges that need to be addressed.

References

[1] Rui Abreu, João Paulo Fernandes, Luis Llana, and Guilherme Tavares. 2022. Metamorphic testing of oracle quantum programs. In *Proceedings of the 3rd International Workshop on Quantum Software Engineering*, 16–23.

[2] Rajeev Acharya, Laleh Aghababaie-Beni, Igor Aleiner, Trond I. Andersen, Markus Ansmann, Frank Arute, Kunal Arya, Abraham Asfaw, Nikita Astrakhantsev, Juan Atalaya, et al. 2024. Quantum error correction below the surface code threshold. arXiv:2408.13687. Retrieved from https://arxiv.org/abs/2408.13687

[3] Toufique Ahmed and Premkumar Devanbu. 2022. Few-shot training LLMs for project-specific code-summarization. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 1–5.

[4] Shaukat Ali, Paolo Arcaini, Xinyi Wang, and Tao Yue. 2021. Assessing the effectiveness of input and output coverage criteria for testing quantum programs. In *Proceedings of the 2021 14th IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 13–23.

[5] Shaukat Ali, Tao Yue, and Rui Abreu. 2022. When software engineering meets quantum computing. *Communications of the ACM* 65, 4 (2022), 84–88. DOI: https://doi.org/10.1145/3512340

[6] Adriana Alvarado Garcia, Heloisa Candello, Karla Badillo-Urquiola, and Marisol Wong-Villacres. 2025. Emerging data practices: Data work in the era of large language models. *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*. ACM, New York, NY, Article 846, 1–21. DOI: https://doi.org/10.1145/3706598.3714069

[7] Jaime Alvarado-Valiente, Javier Romero-Álvarez, Enrique Moguel, Jose García-Alonso, and Juan M. Murillo. 2024. Orchestration for quantum services: The power of load balancing across multiple service providers. *Science of Computer Programming* 237 (2024), 103139.

[8] Niyazi Furkan Bar and Mehmet Karakose. 2024. Quantum circuit dataset generator approach for deep learning based solutions. In *Proceedings of the 2024 28th International Conference on Information Technology (IT)*. IEEE, 1–4.

[9] Bela Bauer, Sergey Bravyi, Mario Motta, and Garnet Kin-Lic Chan. 2020. Quantum algorithms for quantum chemistry and quantum materials science. *Chemical Reviews* 120, 22 (2020), 12685–12717.

[10] Lev S. Bishop. 2017. QASM 2.0: A quantum circuit intermediate representation. In *Proceedings of the APS March Meeting Abstracts*, P46-008. Retrieved from https://meetings-archive.aps.org/mar/2017/p46/8/

[11] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*. H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 1877–1901. Retrieved from https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfcb4967418bfb8ac142f64a-Paper.pdf

[12] Marcello Caleffi, Michele Amoretti, Davide Ferrari, Jessica Illiano, Antonio Manzalini, and Angela Sara Cacciapuoti. 2024. Distributed quantum computing: A survey. *Computer Networks* 254 (2024), 110672.

[13] Charlie Campbell, Hao Mark Chen, Wayne Luk, and Hongxiang Fan. 2025. Enhancing LLM-based quantum code generation with multi-agent optimization and quantum error correction. In *Proceedings of the 2025 62nd ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–7.

[14] José Campos and André Souto. 2021. QBugs: A collection of reproducible bugs in quantum algorithms and a supporting infrastructure to enable controlled quantum software testing and debugging experiments. In *Proceedings of the 2021 IEEE/ACM 2nd International Workshop on Quantum Software Engineering (Q-SE)*. IEEE, 28–32.

[15] Pietro Cassieri, Giuseppe Scanniello, Seung Yeob Shin, Fabrizio Pastore, and Domenico Bianculli. 2026. Beyond rules: LLM-powered linting for quantum programs. arXiv:2605.03943. Retrieved from https://arxiv.org/abs/2605.03943

[16] Eason Chen, Ray Huang, Han-Shin Chen, Yuen-Hsien Tseng, and Liang-Yi Li. 2023. GPTutor: A ChatGPT-powered programming tool for code explanation. In *Proceedings of the International Conference on Artificial Intelligence in Education*. Springer, 321–327.

[17] Kean Chen, Wang Fang, Ji Guan, Xin Hong, Mingyu Huang, Junyi Liu, Qisheng Wang, and Mingsheng Ying. 2022. VeriQBench: A benchmark for multiple types of quantum circuits. arXiv:2206.10880. Retrieved from https://arxiv.org/abs/2206.10880

[18] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. arXiv:2107.03374. Retrieved from https://arxiv.org/abs/2107.03374

[19] QiHong Chen, Jiachen Yu, Jiawei Li, Jiecheng Deng, Justin Tian Jin Chen, and Iftekhar Ahmed. 2025. A deep dive into large language model code generation mistakes: What and why? arXiv:2411.01414. Retrieved from https://arxiv.org/abs/2411.01414

[20] Yinghao Chen, Zehao Hu, Chen Zhi, Junxiao Han, Shuiguang Deng, and Jianwei Yin. 2024. Chatunitest: A framework for LLM-based test generation. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, 572–576.

[21] Frederic T. Chong, Diana Franklin, and Margaret Martonosi. 2017. Programming languages and compiler design for realistic quantum hardware. *Nature* 549, 7671 (2017), 180–187.

[22] Giordano d'Aloisio, Sophie Fortz, Carol Hanna, Daniel Fortunato, Avner Bensoussan, Eñaut Mendiluze Usandizaga, and Federica Sarro. 2024. Exploring LLM-Driven explanations for quantum algorithms. In *Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, 475–481.

[23] Cirq Developers. 2024. *Cirq*. Zenodo. DOI: https://doi.org/10.5281/ZENODO.4062499

[24] Nivedita Dey, Mrityunjay Ghosh, Subhra Samir Kundu, and Amlan Chakrabarti, et al. 2020. QDLC—The quantum development life cycle. arXiv:2010.08053. Retrieved from https://arxiv.org/abs/2010.08053

[25] Olivia Di Matteo, Santiago Núñez-Corrales, Michał Stęchły, Steven P. Reinhardt, and Tim Mattson. 2024. An abstraction hierarchy toward productive quantum programming. In *Proceedings of the 2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*, Vol. 1, 979–989. DOI: https://doi.org/10.1109/QCE60285.2024.00117

[26] Edsger W. Dijkstra. 1978. On the foolishness of "natural language programming". In *Program Construction, International Summer School*. Springer-Verlag, Berlin, 51–53.

[27] Bosheng Ding, Chengwei Qin, Ruochen Zhao, Tianze Luo, Xinze Li, Guizhen Chen, Wenhan Xia, Junjie Hu, Anh Tuan Luu, and Shafiq Joty. 2024. Data augmentation using LLMs: Data perspectives, learning paradigms and challenges. In *Proceedings of the Findings of the Association for Computational Linguistics (ACL '24)*. Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.), Association for Computational Linguistics, 1679–1705. DOI: https://doi.org/10.18653/v1/2024.findings-acl.97

[28] Tuan Dinh, Jinman Zhao, Samson Tan, Renato Negrinho, Leonard Lausen, Sheng Zha, and George Karypis. 2023. Large language models of code fail at completing code with potential bugs. In *Proceedings of the 37th Conference on Neural Information Processing Systems*. Retrieved from https://openreview.net/forum?id=e1WgjvFGWp

[29] Yuxuan Du, Tao Huang, Shan You, Min-Hsiu Hsieh, and Dacheng Tao. 2022. Quantum circuit architecture search for variational quantum algorithms. *Npj Quantum Information* 8, 1 (2022), 62.

[30] Nicolas Dupuis, Luca Buratti, Sanjay Vishwakarma, Aitana Viudes Forrat, David Kremer, Ismael Faro, Ruchir Puri, and Juan Cruz-Benito. 2024. Qiskit code assistant: Training LLMs for generating quantum computing code. arXiv:2405.19495. Retrieved from https://arxiv.org/abs/2405.19495

[31] Pau Escofet, Santiago Rodrigo, Artur Garcia-Sáez, Eduard Alarcón, Sergi Abadal, and Carmen G. Almudéver. 2025. An accurate and efficient analytic model of fidelity under depolarizing noise oriented to large scale quantum system design. *Quantum Science and Technology* 10, 3 (Jul. 2025), 035061. DOI: https://doi.org/10.1088/2058-9565/aded2f

[32] Olivier Ezratty. 2023. How AI, LLMs and Quantum Science Can Empower Each Other. Retrieved from https://www.oezratty.net/Files/Publications/How%20LLMs%20and%20quantum%20science%20can%20empower%20each%20other.pdf

[33] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M. Zhang. 2023. Large language models for software engineering: Survey and open problems. In *Proceedings of the 2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*. IEEE, 31–53.

[34] Miriam Fernández-Osuna, Ricardo Pérez-Castillo, José A. Cruz-Lemus, Michal Baczyk, and Mario Piattini. 2025. Exploring design patterns in quantum software: A case study. *Computing* 107, 5 (2025), 1–31.

[35] Alessio Ferrari and Paola Spoletini. 2025. Formal requirements engineering and large language models: A two-way roadmap. *Information and Software Technology* 181 (2025), 107697.

[36] Daniel Fortunato, José Campos, and Rui Abreu. 2024. Gate branch coverage: A metric for quantum software testing. In *Proceedings of the 1st ACM International Workshop on Quantum Software Engineering: The Next Evolution*, 15–18.

[37] Mohammad Fraiwan and Natheer Khasawneh. 2023. A review of ChatGPT applications in education, marketing, software engineering, and healthcare: Benefits, drawbacks, and research directions. arXiv:2305.00237. Retrieved from https://arxiv.org/abs/2305.00237

[38] Zhenxiao Fu, Fan Chen, and Lei Jiang. 2025. QAgent: An LLM-based multi-agent system for autonomous OpenQASM programming. arXiv:2508.20134. Retrieved from https://arxiv.org/abs/2508.20134

[39] Jose Garcia-Alonso, Javier Rojo, David Valencia, Enrique Moguel, Javier Berrocal, and Juan Manuel Murillo. 2021. Quantum software as a service through a quantum API gateway. *IEEE Internet Computing* 26, 1 (2021), 34–41.

[40] Felix Gemeinhardt, Antonio Garmendia, and Manuel Wimmer. 2021. Towards model-driven quantum software engineering. In *Proceedings of the 2021 IEEE/ACM 2nd International Workshop on Quantum Software Engineering (Q-SE)*. IEEE, 13–15.

[41] Alexander S. Green, Peter LeFanu Lumsdaine, Neil J. Ross, Peter Selinger, and Benoît Valiron. 2013. Quipper: A scalable quantum programming language. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 333–342.

[42] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Yu Wu, Y. K. Li, et al. 2024. DeepSeek-Coder: When the large language model meets programming–The rise of code intelligence. arXiv:2401.14196. Retrieved from https://arxiv.org/abs/2401.14196

[43] Xiaoyu Guo, Takahiro Muta, and Jianjun Zhao. 2024. Quantum circuit ansatz: Patterns of abstraction and reuse of quantum algorithm design. In *Proceedings of the 2024 IEEE International Conference on Quantum Software (QSW)*. IEEE, 69–80.

[44] Xiaoyu Guo, Shinobu Saito, and Jianjun Zhao. 2025. M2QCode: A model-driven framework for generating multi-platform quantum programs. In *Proceedings of the 2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE Press, 3639–3649. DOI: https://doi.org/10.1109/ASE63991.2025.00301

[45] Xiaoyu Guo, Shinobu Saito, and Jianjun Zhao. 2025. QuanUML: Towards a modeling language for model-driven quantum software development. In *Proceedings of the 2025 IEEE 49th Annual Computers, Software, and Applications Conference (COMPSAC)*. IEEE, 1344–1349.

[46] Xiaoyu Guo, Minggu Wang, and Jianjun Zhao. 2025. QuanBench: Benchmarking quantum code generation with large language models. In *Proceedings of the 2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE Press, 2657–2669. DOI: https://doi.org/10.1109/ASE63991.2025.00218

[47] Xiaoyu Guo, Jianjun Zhao, and Pengzhan Zhao. 2024. On repairing quantum programs using ChatGPT. In *Proceedings of the 5th ACM/IEEE International Workshop on Quantum Software Engineering*, 9–16.

[48] Andreas Happe and Jürgen Cito. 2023. Getting pwn'd by AI: Penetration testing with large language models. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2082–2086.

[49] Elena R. Henderson, Jessie M. Henderson, Joshua Ange, and Mitchell A. Thornton. 2025. Programming quantum computers with large language models. In *Proceedings of the Quantum Information Science, Sensing, and Computation XVII*, Vol. 13451. SPIE, 20–34.

[50] Shahin Honarvar, Mohammad Reza Mousavi, and Rajagopal Nagarajan. 2020. Property-based testing of quantum programs in Q#. In *Proceedings of the IEEE/ACM 42nd International Conference on Software Engineering Workshops*, 430–435.

[51] Qing Huang, Zhenyu Wan, Zhenchang Xing, Changjing Wang, Jieshan Chen, Xiwei Xu, and Qinghua Lu. 2023. Let's chat to find the APIs: Connecting human, LLM and knowledge graph through AI chain. In *Proceedings of the 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 471–483.

[52] Yipeng Huang and Margaret Martonosi. 2019. Statistical assertions for validating patterns and finding bugs in quantum programs. In *Proceedings of the 46th International Symposium on Computer Architecture*, 541–553.

[53] Raban Iten, Romain Moyard, Tony Metger, David Sutter, and Stefan Woerner. 2022. Exact and practical pattern matching for quantum circuit optimization. *ACM Transactions on Quantum Computing* 3, 1 (2022), 1–41.

[54] Linus Jern, Valter Uotila, Cong Yu, and Bo Zhao. 2025. Agent-Q: Fine-tuning large language models for quantum circuit generation and optimization. In *Proceedings of the 2025 IEEE International Conference on Quantum Computing and Engineering (QCE). IEEE*.

[55] Tiancheng Jin, Shangzhou Xia, and Jianjun Zhao. 2025. NovaQ: Improving quantum program testing through Diversity-Guided test case generation. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering*, 3881–3885. DOI: https://doi.org/10.1109/ASE63991.2025.00335

[56] Eric R. Johnston, Nic Harrigan, and Mercedes Gimeno-Segovia. 2019. *Programming Quantum Computers: Essential Algorithms and Code Samples*. O'Reilly Media.

[57] Sungmin Kang, Bei Chen, Shin Yoo, and Jian-Guang Lou. 2025. Explainable automated debugging via large language model-driven scientific debugging. *Empirical Software Engineering* 30, 2 (2025), 45.

[58] Arif Ali Khan, Muhammad Azeem Akbar, Aakash Ahmad, Mahdi Fahmideh, Mohammad Shameem, Valtteri Lahtinen, Muhammad Waseem, and Tommi Mikkonen. 2023. Agile practices for quantum software development: Practitioners' perspectives. In *Proceedings of the 2023 IEEE International Conference on Quantum Software (QSW)*. IEEE, 9–20.

[59] Mansoor A. Khan, Muhammad N. Aman, and Biplab Sikdar. 2024. Beyond bits: A review of quantum embedding techniques for efficient information processing. *IEEE Access* 12 (2024), 46118–46137.

[60] Andrew Kiruluta. 2025. Quantum circuit reasoning models: A variational framework for differentiable logical inference. arXiv:2512.07871. Retrieved from https://arxiv.org/abs/2512.07871

[61] Timo Aukusti Laine. 2025. The quantum LLM: Modeling semantic spaces with quantum principles. arXiv:2504.13202. Retrieved from https://arxiv.org/abs/2504.13202

[62] Yu-Hong Lam, Yuriy Abramov, Ravi S. Ananthula, Jennifer M. Elward, Lori R. Hilden, Sten O. Nilsson Lill, Per-Ola Norrby, Antonio Ramirez, Edward C. Sherer, Jason Mustakis, et al. 2020. Applications of quantum chemistry in pharmaceutical process development: Current state and opportunities. *Organic Process Research and Development* 24, 8 (2020), 1496–1507.

[63] Alexander LeClair, Sakib Haque, Lingfei Wu, and Collin McMillan. 2020. Improved code summarization via a graph neural network. In *Proceedings of the 28th International Conference on Program Comprehension*, 184–195.

[64] Ang Li, Samuel Stein, Sriram Krishnamoorthy, and James Ang. 2023. QASMBench: A low-level quantum benchmark suite for NISQ evaluation and simulation. *ACM Transactions on Quantum Computing* 4, 2 (2023), 1–26.

[65] Gushu Li, Li Zhou, Nengkun Yu, Yufei Ding, Mingsheng Ying, and Yuan Xie. 2020. Projection-based runtime assertions for testing and debugging quantum programs. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (2020), 1–29.

[66] Xin-Ye Li, Jiang-Tian Xue, Zheng Xie, and Ming Li. 2023. Think outside the code: Brainstorming boosts large language models in code generation. arXiv:2305.10679. Retrieved from https://arxiv.org/abs/2305.10679

[67] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. 2022. Competition-level code generation with alphacode. *Science* 378, 6624 (2022), 1092–1097.

[68] Yuechen Li, Hanyu Pei, Linzhi Huang, Beibei Yin, and Kai-Yuan Cai. 2024. Automatic repair of quantum programs via unitary operation. *ACM Transactions on Software Engineering and Methodology* 33, 6 (2024), 1–43.

[69] Yangjia Li and Mingsheng Ying. 2014. Debugging quantum processes using monitoring measurements. *Physical Review A* 89, 4 (2014), 042338.

[70] Zhuoyan Li, Hangxiao Zhu, Zhuoran Lu, and Ming Yin. 2023. Synthetic data generation with large language models for text classification: Potential and limitations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Retrieved from https://openreview.net/forum?id=MmBjKmHIND

[71] Zhiding Liang, Jinglei Cheng, Rui Yang, Hang Ren, Zhixin Song, Di Wu, Xuehai Qian, Tongyang Li, and Yiyu Shi. 2023. Unleashing the potential of LLMs for quantum computing: A study in quantum architecture design. arXiv:2307.08191. Retrieved from https://arxiv.org/abs/2307.08191

[72] Zhan Ling, Yunhao Fang, Xuanlin Li, Zhiao Huang, Mingu Lee, Roland Memisevic, and Hao Su. 2023. Deductive verification of chain-of-thought reasoning. In *Advances in Neural Information Processing Systems*, Vol. 36, 36407–36433.

[73] Ji Liu, Gregory T. Byrd, and Huiyang Zhou. 2020. Quantum circuits for dynamic runtime assertions in quantum computation. In *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems*, 1017–1030.

[74] Yue Liu, Thanh Le-Cong, Ratnadira Widyasari, Chakkrit Tantithamthavorn, Li Li, Xuan-Bach D. Le, and David Lo. 2024. Refining ChatGPT-generated code: Characterizing and mitigating code quality issues. *ACM Transactions on Software Engineering and Methodology* 33, 5 (2024), 1–26.

[75] Peixun Long and Jianjun Zhao. 2024. Equivalence, identity, and unitarity checking in black-box testing of quantum programs. *Journal of Systems and Software* 211 (2024), 112000.

[76] Peixun Long and Jianjun Zhao. 2024. Testing multi-subroutine quantum programs: From unit testing to integration testing. *ACM Transactions on Software Engineering and Methodology* 33, 6, Article 147 (2024), 1–61.

[77] Peixun Long and Jianjun Zhao. 2025. A black-box testing framework for oracle quantum programs. arXiv:2505.07243. Retrieved from https://arxiv.org/abs/2505.07243

[78] Guilong Lu, Xiaolin Ju, Xiang Chen, Wenlong Pei, and Zhilong Cai. 2024. GRACE: Empowering LLM-based software vulnerability detection with graph structure and in-context learning. *Journal of Systems and Software* 212 (2024), 112031.

[79] Junjie Luo and Jianjun Zhao. 2025. Formalization of quantum intermediate representations for code safety. *Journal of Systems and Software* 219 (Jan. 2025), 112236. DOI: https://doi.org/10.1016/j.jss.2024.112236

[80] Junjie Luo, Pengzhan Zhao, Zhongtao Miao, Shuhan Lan, and Jianjun Zhao. 2022. A comprehensive study of bug fixes in quantum programs. In *Proceedings of the 2022 IEEE International Conference on Software Analysis, Evolution and Reengineering*. IEEE, 1239–1246. DOI: https://doi.org/10.1109/SANER53432.2022.00147

[81] Yang Luo, Richard Yu, Fajun Zhang, Ling Liang, and Yongqiang Xiong. 2024. Bridging gaps in LLM code translation: Reducing errors with call graphs and bridged debuggers. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2448–2449.

[82] Roman Macháček, Anastasiia Grishina, Max Hort, and Leon Moonen. 2025. The impact of fine-tuning large language models on automated program repair. In *Proceedings of the 2025 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 380–392. DOI: https://doi.org/10.1109/ICSME64153.2025.00042

[83] Xuan Mao, Zijie Huang, Jianxin Ge, Chao Wang, Wuxu Wang, and Lizhi Cai. 2025. Towards defect prediction for quantum software. In *Proceedings of the 2025 IEEE/ACM International Workshop on Quantum Software Engineering (Q-SE)*. IEEE, 1–8.

[84] Antonio Mastropaolo. 2025. Prompt alchemy: Engineering the magic of code. *Computer* 58, 12 (Dec. 2025), 18–20. DOI: https://doi.org/10.1109/MC.2025.3615835

[85] Sara Ayman Metwalli and Rodney Van Meter. 2024. Testing and debugging quantum circuits. *IEEE Transactions on Quantum Engineering* 5 (2024), 1–15.

[86] Taku Mikuriya, Tatsuya Ishigaki, Masayuki Kawarada, Shunya Minami, Tadashi Kadowaki, Yohichi Suzuki, Soshun Naito, Shunya Takada, Takumi Kato, Tamotsu Basseda, et al. 2025. QCoder benchmark: Bridging language generation and quantum hardware through Simulator-Based feedback. In *Proceedings of the 18th International Natural Language Generation Conference*, 743–752.

[87] Andriy Miranskyy. 2025. The cost of certainty: Shot budgets in quantum program testing. arXiv:2510.22418. Retrieved from https://arxiv.org/abs/2510.22418

[88] Andriy Miranskyy and Lei Zhang. 2019. On testing quantum programs. In *Proceedings of the 2019 IEEE/ACM 41st International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*. IEEE, 57–60.

[89] Andriy Miranskyy, Lei Zhang, and Javad Doliskani. 2020. Is your quantum program bug-free? In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER '20)*. ACM, New York, NY, 29–32. DOI: https://doi.org/10.1145/3377816.3381731

[90] Andriy Miranskyy, Lei Zhang, and Javad Doliskani. 2021. On testing and debugging quantum software. arXiv:2103.09172. Retrieved from https://arxiv.org/abs/2103.09172

[91] Mohammad Mahdi Mohajer, Reem Aleithan, Nima Shiri Harzevili, Moshi Wei, Alvine Boaye Belle, Hung Viet Pham, and Song Wang. 2023. SkipAnalyzer: A tool for static code analysis with large language models. arXiv:2310.18532. Retrieved from https://arxiv.org/abs/2310.18532

[92] Viraaji Mothukuri, Reza M. Parizi, and James L. Massa. 2024. LLMSmartSec: Smart contract security auditing with LLM and annotated control flow graph. In *Proceedings of the 2024 IEEE International Conference on Blockchain (Blockchain)*. IEEE, 434–441.

[93] Asmar Muqeet, Tao Yue, Shaukat Ali, and Paolo Arcaini. 2024. Mitigating noise in quantum software testing using machine learning. *IEEE Transactions on Software Engineering* 50, 11 (2024), 2947–2961.

[94] Juan Manuel Murillo, Jose Garcia-Alonso, Enrique Moguel, Johanna Barzen, Frank Leymann, Shaukat Ali, Tao Yue, Paolo Arcaini, Ricardo Pérez-Castillo, Ignacio García-Rodríguez de Guzmán, et al. 2025. Quantum software engineering: Roadmap and challenges ahead. *ACM Transactions on Software Engineering and Methodology* 34, 5, Article 154 (May 2025), 1–48. DOI: https://doi.org/10.1145/3712002

[95] Akimoto Nakayama, Kosuke Mitarai, Leonardo Placidi, Takanori Sugimoto, and Keisuke Fujii. 2025. VQE-generated quantum circuit dataset for machine learning. *Physical Review Research* 7, 3 (2025), 033048.

[96] Daye Nam, Andrew Macvean, Vincent Hellendoorn, Bogdan Vasilescu, and Brad Myers. 2024. Using an LLM to help with code understanding. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 1–13.

[97] Paul D. Nation, Abdullah Ash Saki, Sebastian Brandhofer, Luciano Bello, Shelly Garion, Matthew Treinish, and Ali Javadi-Abhari. 2024. Benchmarking the performance of quantum computing software. arXiv:2409.08844. Retrieved from https://arxiv.org/abs/2409.08844

[98] Humza Naveed, Asad Ullah Khan, Shi Qiu, Muhammad Saqib, Saeed Anwar, Muhammad Usman, Naveed Akhtar, Nick Barnes, and Ajmal Mian. 2025. A comprehensive overview of large language models. *ACM Transactions on Intelligent Systems and Technology* 16, 5 (2025), 1–72.

[99] OpenAI. 2023. ChatGPT (GPT-4). Retrieved February 1, 2025 from https://chat.openai.com/

[100] Scott Pakin. 2016. A quantum macro assembler. In *Proceedings of the 2016 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE, 1–8.

[101] Matteo Paltenghi and Michael Pradel. 2022. Bugs in quantum computing platforms: An empirical study. *Proceedings of the ACM on Programming Languages* 6, OOPSLA1 (2022), 1–27.

[102] Matteo Paltenghi and Michael Pradel. 2023. MorphQ: Metamorphic testing of the Qiskit quantum computing platform. In *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2413–2424.

[103] Matteo Paltenghi and Michael Pradel. 2024. Analyzing quantum programs with LintQ: A static analysis framework for Qiskit. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 2144–2166.

[104] Rangeet Pan, Ali Reza Ibrahimzada, Rahul Krishna, Divya Sankar, Lambert Pouguem Wassi, Michele Merler, Boris Sobolev, Raju Pavuluri, Saurabh Sinha, and Reyhaneh Jabbarvand. 2023. Understanding the effectiveness of large language models in code translation. arXiv:2308.03109v1. Retrieved from https://arxiv.org/abs/2308.03109v1

[105] Michael P. Papazoglou, Paolo Traverso, Schahram Dustdar, and Frank Leymann. 2007. Service-oriented computing: State of the art and research challenges. *Computer* 40, 11 (2007), 38–45.

[106] Ricardo Pérez-Castillo and Mario Piattini. 2022. Design of classical-quantum systems with UML. *Computing* 104, 11 (2022), 2375–2403.

[107] Carlos A. Pérez-Delgado and Hector G. Perez-Gonzalez. 2020. Towards a quantum software modeling language. In *Proceedings of the IEEE/ACM 42nd International Conference on Software Engineering Workshops*, 442–444.

[108] Alberto Peruzzo, Jarrod McClean, Peter Shadbolt, Man-Hong Yung, Xiao-Qi Zhou, Peter J. Love, Alán Aspuru-Guzik, and Jeremy L. O'Brien. 2014. A variational eigenvalue solver on a photonic quantum processor. *Nature Communications* 5, 1 (2014), 4213.

[109] Mario Piattini, Manuel Serrano, Ricardo Perez-Castillo, Guido Petersen, and Jose Luis Hevia. 2021. Toward a quantum software engineering. *IT Professional* 23, 1 (2021), 62–66.

[110] Stefano Pirandola, Ulrik L. Andersen, Leonardo Banchi, Mario Berta, Darius Bunandar, Roger Colbeck, Dirk Englund, Tobias Gehring, Cosmo Lupo, Carlo Ottaviani, et al. 2020. Advances in quantum cryptography. *Advances in Optics and Photonics* 12, 4 (2020), 1012–1236.

[111] Leonardo Placidi, Ryuichiro Hataya, Toshio Mori, Koki Aoyama, Hayata Morisaki, Kosuke Mitarai, and Keisuke Fujii. 2023. MNISQ: A large-scale quantum circuit dataset for machine learning on/for quantum computers in the NISQ era. arXiv:2306.16627. Retrieved from https://arxiv.org/abs/2306.16627

[112] Khushboo Qayyum, Muhammad Hassan, Sallar Ahmadi-Pour, Chandan Kumar Jha, and Rolf Drechsler. 2024. Late breaking results: LLM-assisted automated incremental proof generation for hardware verification. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 1–2.

[113] Nils Quetschlich, Lukas Burgholzer, and Robert Wille. 2023. MQT bench: Benchmarking software and design automation tools for quantum computing. *Quantum* 7 (2023), 1062.

[114] Javier Romero-Álvarez, Jaime Alvarado-Valiente, Enrique Moguel, and Jose Garcia-Alonso. 2023. Quantum web services: Development and deployment. In *Proceedings of the International Conference on Web Engineering*. Springer, 421–423.

[115] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. 2023. Code llama: Open foundation models for code. arXiv:2308.12950. Retrieved from https://arxiv.org/abs/2308.12950

[116] Prateek Sancheti, Kamalakar Karlapalem, and Kavita Vemuri. 2024. LLM driven web profile extraction for identical names. In *Companion Proceedings of the ACM on Web Conference 2024*, 1616–1625.

[117] Naoto Sato and Ryota Katsube. 2024. Locating buggy segments in quantum program debugging. In *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results*, 26–31.

[118] Melika Sepidband, Hamed Taherkhani, Song Wang, and Hadi Hemmati. 2025. Enhancing LLM-Based code generation with complexity metrics: A feedback-driven approach. In *Proceedings of the 2025 IEEE 49th Annual Computers, Software, and Applications Conference (COMPSAC)*. IEEE Computer Society, 1416–1426. DOI: https://doi.org/10.1109/COMPSAC65507.2025.00178

[119] Minqi Shao and Jianjun Zhao. 2024. A coverage-guided testing framework for quantum neural networks. arXiv:2411.02450. Retrieved from https://arxiv.org/abs/2411.02450

[120] Julian Shen, Joshua Ammermann, Christoph König, and Ina Schaefer. 2025. Quantum pattern detection: Accurate state-and circuit-based analyses. In *Proceedings of the 2025 IEEE/ACM International Workshop on Quantum Software Engineering (Q-SE)*. IEEE, 9–16.

[121] Seung Yeob Shin, Fabrizio Pastore, and Domenico Bianculli. 2025. Quantum program linting with LLMs: Emerging results from a comparative study. In *Proceedings of the 2025 IEEE International Conference on Quantum Computing and Engineering (QCE)*. IEEE Computer Society, 181–186. DOI: https://doi.org/10.1109/QCE65121.2025.10316

[122] Nazanin Siavash and Armin Moin. 2025. LLM-powered quantum code transpilation. In *Proceedings of the 2025 IEEE International Conference on Quantum Computing and Engineering (QCE)*, Vol. 2. IEEE, 410–411.

[123] Nazanin Siavash and Armin Moin. 2025. Model-driven quantum code generation using large language models and Retrieval-Augmented generation. In *Proceedings of the 2025 ACM/IEEE 28th International Conference on Model Driven Engineering Languages and Systems (MODELS)*, 260–266. DOI: https://doi.org/10.1109/MODELS67397.2025.00031

[124] Yisheng Song, Ting Wang, Puyu Cai, Subrota K. Mondal, and Jyoti Prakash Sahoo. 2023. A comprehensive survey of few-shot learning: Evolution, applications, challenges, and opportunities. *ACM Computing Surveys* 55, 13s, Article 271 (Jul. 2023), 1–40. DOI: https://doi.org/10.1145/3582688

[125] Krysta Svore, Alan Geller, Matthias Troyer, John Azariah, Christopher Granade, Bettina Heim, Vadym Kliuchnikov, Mariia Mykhailova, Andres Paz, and Martin Roetteler. 2018. Q# enabling scalable quantum computing and development with a high-level DSL. In *Proceedings of the Real World Domain Specific Languages Workshop 2018*, 1–10.

[126] Runzhou Tao. 2024. *Formal Verification of Quantum Software*. Columbia University.
[127] Krishna Upadhyay, Vinaik Chhetri, A. B. Siddique, and Umar Farooq. 2025. Analyzing the evolution and maintenance of quantum software repositories. In *Proceedings of the 2025 IEEE International Conference on Quantum Software (QSW)*. IEEE Computer Society, 173–184. DOI: https://doi.org/10.1109/QSW67625.2025.00029
[128] David Valencia, Jose Garcia-Alonso, Javier Rojo, Enrique Moguel, Javier Berrocal, and Juan Manuel Murillo. 2021. Hybrid classical-quantum software services systems: Exploration of the rough edges. In *Proceedings of the International Conference on the Quality of Information and Communications Technology*. Springer, 225–238.
[129] Sanjay Vishwakarma, Francis Harkins, Siddharth Golecha, Vishal Sharathchandra Bajpe, Nicolas Dupuis, Luca Buratti, David Kremer, Ismael Faro, Ruchir Puri, and Juan Cruz-Benito. 2024. Qiskit HumanEval: An evaluation benchmark for quantum code generative models. In *Proceedings of the 2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*, Vol. 1. IEEE, 1169–1176.
[130] Haoran Wang, Pingzhi Li, Min Chen, Jinglei Cheng, Junyu Liu, and Tianlong Chen. 2024. GroverGPT: A large language model with 8 billion parameters for quantum searching. arXiv:2501.00135. Retrieved from https://arxiv.org/abs/2501.00135
[131] Jiyuan Wang, Ming Gao, Yu Jiang, Jianguang Lou, Yue Gao, Dongmei Zhang, and Jiaguang Sun. 2018. QuanFuzz: Fuzz testing of quantum program. arXiv:1810.10310. Retrieved from https://arxiv.org/abs/1810.10310
[132] Jiyuan Wang, Qian Zhang, Guoqing Harry Xu, and Miryung Kim. 2021. QDiff: Differential testing of quantum software stacks. In *Proceedings of the 2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 692–704.
[133] Xinyi Wang, Paolo Arcaini, Tao Yue, and Shaukat Ali. 2021. Application of combinatorial testing to quantum programs. In *Proceedings of the 2021 IEEE 21st International Conference on Software Quality, Reliability and Security (QRS)*. IEEE, 179–188.
[134] Xinyi Wang, Paolo Arcaini, Tao Yue, and Shaukat Ali. 2021. Generating failing test suites for quantum programs with search. In *Proceedings of the International Symposium on Search Based Software Engineering*. Springer, 9–25.
[135] Xinyi Wang, Paolo Arcaini, Tao Yue, and Shaukat Ali. 2021. Quito: A coverage-guided test generator for quantum programs. In *Proceedings of the 2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1237–1241.
[136] Xinyi Wang, Paolo Arcaini, Tao Yue, and Shaukat Ali. 2022. QuSBT: Search-based testing of quantum programs. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*, 173–177.
[137] Xinyi Wang, Paolo Arcaini, Tao Yue, and Shaukat Ali. 2023. QuCAT: A combinatorial testing tool for quantum software. In *Proceedings of the 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2066–2069.
[138] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. 2024. Executable code actions elicit better LLM agents. In *Proceedings of the 41 st International Conference on Machine Learning*. Retrieved from https://openreview.net/forum?id=jJ9BoXAfFa
[139] Benjamin Weder, Johanna Barzen, Frank Leymann, and Daniel Vietz. 2022. Quantum software development lifecycle. In *Quantum Software Engineering*. Manuel A. Serrano, Ricardo Pérez-Castillo, and Mario Piattini (Eds.), Springer, 61–83.
[140] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V. Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, Vol. 35, 24824–24837.
[141] Robert Wille, Rod Van Meter, and Yehuda Naveh. 2019. IBM's Qiskit tool chain: Working with and developing for real quantum computers. In *Proceedings of the 2019 Design, Automation and Test in Europe Conference and Exhibition (DATE)*. IEEE, 1234–1240.
[142] William K. Wootters and Wojciech H. Zurek. 1982. A single quantum cannot be cloned. *Nature* 299, 5886 (1982), 802–803.
[143] Ewa Woźny, Jarosław Hryszko, and Adam Roman. 2025. Leveraging large language models for software defect detection. In *Proceedings of the 51st Euromicro Conference on Software Engineering and Advanced Applications (SEAA '25)*, 125–142.
[144] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. 2023. Automated program repair in the era of large pre-trained language models. In *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1482–1494.
[145] Shangzhou Xia and Jianjun Zhao. 2023. Static entanglement analysis of quantum programs. In *Proceedings of the 2023 IEEE/ACM 4th International Workshop on Quantum Software Engineering (Q-SE)*. IEEE, 42–49.
[146] Shangzhou Xia, Jianjun Zhao, Fuyuan Zhang, and Xiaoyu Guo. 2025. Quantum concolic testing. *Proceedinds of the ACM on Software Engineering* 2, ISSTA, Article ISSTA051 (Jun. 2025), 1–21. DOI: https://doi.org/10.1145/3728926

[147] Shuyuan Xu, Zelong Li, Kai Mei, and Yongfeng Zhang. 2024. CoRE: LLM as interpreter for natural language programming, pseudo-code programming, and flow programming of AI agents. arXiv:2405.06907. Retrieved from https://arxiv.org/abs/2405.06907

[148] Boyang Yang, Haoye Tian, Jiadong Ren, Hongyu Zhang, Jacques Klein, Tegawendé F. Bissyandé, Claire Le Goues, and Shunfu Jin. 2026. MORepair: Teaching LLMs to repair code via multi-objective fine-tuning. *ACM Transactions on Software Engineering and Methodology* 35, 2, Article 38 (Jan. 2026), 1–38. DOI: https://doi.org/10.1145/3735129

[149] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Press Ofir. 2024. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, Vol. 37, 50528–50652.

[150] Ke Yang, Jiateng Liu, John Wu, Chaoqi Yang, Yi Fung, Sha Li, Zixuan Huang, Xu Cao, Xingyao Wang, Heng Ji, and Chengxiang Zhai. 2024. If LLM is the wizard, then code is the wand: A survey on how code empowers large language models to serve as intelligent agents. In *Proceedings of the ICLR 2024 Workshop on Large Language Model (LLM) Agents*. Retrieved from https://openreview.net/forum?id=8dmNOD9hbq

[151] Zheyuan Yang, Zexi Kuang, Xue Xia, and Yilun Zhao. 2025. Can LLMs generate high-quality test cases for algorithm problems? TestCase-Eval: A systematic evaluation of fault coverage and exposure. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.), Association for Computational Linguistics, 1050–1063. DOI: https://doi.org/10.18653/v1/2025.acl-short.82

[152] Jiaming Ye, Xiongfei Wu, Shangzhou Xia, Fuyuan Zhang, and Jianjun Zhao. 2025. Is measurement enough? Rethinking output validation in quantum program testing. arXiv:2509.16595. Retrieved from https://arxiv.org/abs/2509.16595

[153] Jiaming Ye, Shangzhou Xia, Fuyuan Zhang, Paolo Arcaini, Lei Ma, Jianjun Zhao, and Fuyuki Ishikawa. 2023. QuraTest: Integrating quantum specific features in quantum program testing. In *Proceedings of the 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1149–1161.

[154] Sixiang Ye, Zeyu Sun, Guoqing Wang, Liwei Guo, Qingyuan Liang, Zheng Li, and Yong Liu. 2025. Prompt alchemy: Automatic prompt refinement for enhancing code generation. *IEEE Transactions on Software Engineering* 51, 9 (2025), 2472–2493. DOI: https://doi.org/10.1109/TSE.2025.3589634

[155] Haibo Yu and Jianjun Zhao. 2025. The quantum program dependence graph and its uses in quantum software development. In *Proceedings of the 2025 IEEE/ACM International Workshop on Quantum Software Engineering*. IEEE, 48–55. DOI: https://doi.org/10.1109/Q-SE66736.2025.00014

[156] Yuanhao Yue, Chengyu Wang, Jun Huang, and Peng Wang. 2025. Building a family of data augmentation models for low-cost LLM fine-tuning on the cloud. In *Proceedings of the 31st International Conference on Computational Linguistics: Industry Track*. Owen Rambow, Leo Wanner, Marianna Apidianaki, Hend Al-Khalifa, Barbara Di Eugenio, Steven Schockaert, Kareem Darwish, and Apoorv Agarwal (Eds.). Association for Computational Linguistics, 431–444. Retrieved from https://aclanthology.org/2025.coling-industry.37/

[157] Quanjun Zhang, Chunrong Fang, Yuxiang Ma, Weisong Sun, and Zhenyu Chen. 2023. A survey of learning-based automated program repair. *ACM Transactions on Software Engineering and Methodology* 33, 2 (2023), 1–69.

[158] Jianjun Zhao. 2020. Quantum software engineering: Landscapes and horizons. arXiv:2007.07047. Retrieved from https://arxiv.org/abs/2007.07047

[159] Pengzhan Zhao, Zhongtao Miao, Shuhan Lan, and Jianjun Zhao. 2023. Bugs4Q: A benchmark of existing bugs to enable controlled testing and debugging studies for quantum programs. *Journal of Systems and Software* 205 (2023), 111805.

[160] Pengzhan Zhao, Xiongfei Wu, Zhuo Li, and Jianjun Zhao. 2023. QChecker: Detecting bugs in quantum programs via static analysis. In *Proceedings of the 2023 IEEE/ACM 4th International Workshop on Quantum Software Engineering (Q-SE)*. IEEE, 50–57.

[161] Pengzhan Zhao, Xiongfei Wu, Junjie Luo, Zhuo Li, and Jianjun Zhao. 2023. An empirical study of bugs in quantum machine learning frameworks. In *Proceedings of the 2023 IEEE International Conference on Quantum Software (QSW)*. IEEE, 68–75.

[162] Pengzhan Zhao, Jianjun Zhao, and Lei Ma. 2021. Identifying bug patterns in quantum programs. In *Proceedings of the 2021 IEEE/ACM 2nd International Workshop on Quantum Software Engineering (Q-SE)*. IEEE, 16–21.

[163] Tianyu Zheng, Ge Zhang, Tianhao Shen, Xueling Liu, Bill Yuchen Lin, Jie Fu, Wenhu Chen, and Xiang Yue. 2024. OpenCodeInterpreter: Integrating code generation with execution and refinement. In *Proceedings of the Findings of the Association for Computational Linguistics (ACL '24)*. Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.), Association for Computational Linguistics, 12834–12859. DOI: https://doi.org/10.18653/v1/2024.findings-acl.762

[164] Xin Zhou, Sicong Cao, Xiaobing Sun, and David Lo. 2024. Large language model for vulnerability detection and repair: Literature review and the road ahead. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2024), 1–31.

Received 30 September 2025; revised 29 May 2026; accepted 9 June 2026